

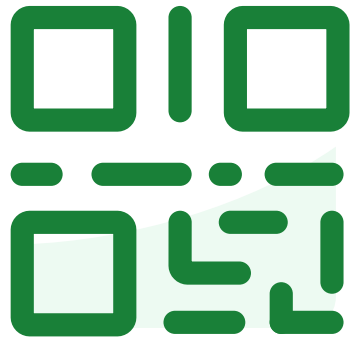
Lecture 7

Linear Regression (2)

Geometric interpretation of least squares and probabilistic view of linear regression

EECS 189/289, Fall 2025 @ UC Berkeley
Joseph E. Gonzalez and Narges Norouzi

Do not edit
How to change the
design



**Join at slido.com
#6101346**

① The Slido app must be installed on every computer you're presenting from

slido

Roadmap

- Error Minimization
- Geometric Interpretation
- Evaluation
- Regularized Least Squares
- When Normal Equation Gets Tricky



Error Minimization

- Error Minimization
- Geometric Interpretation
- Evaluation
- Regularized Least Squares
- When Normal Equation Gets Tricky



6101346

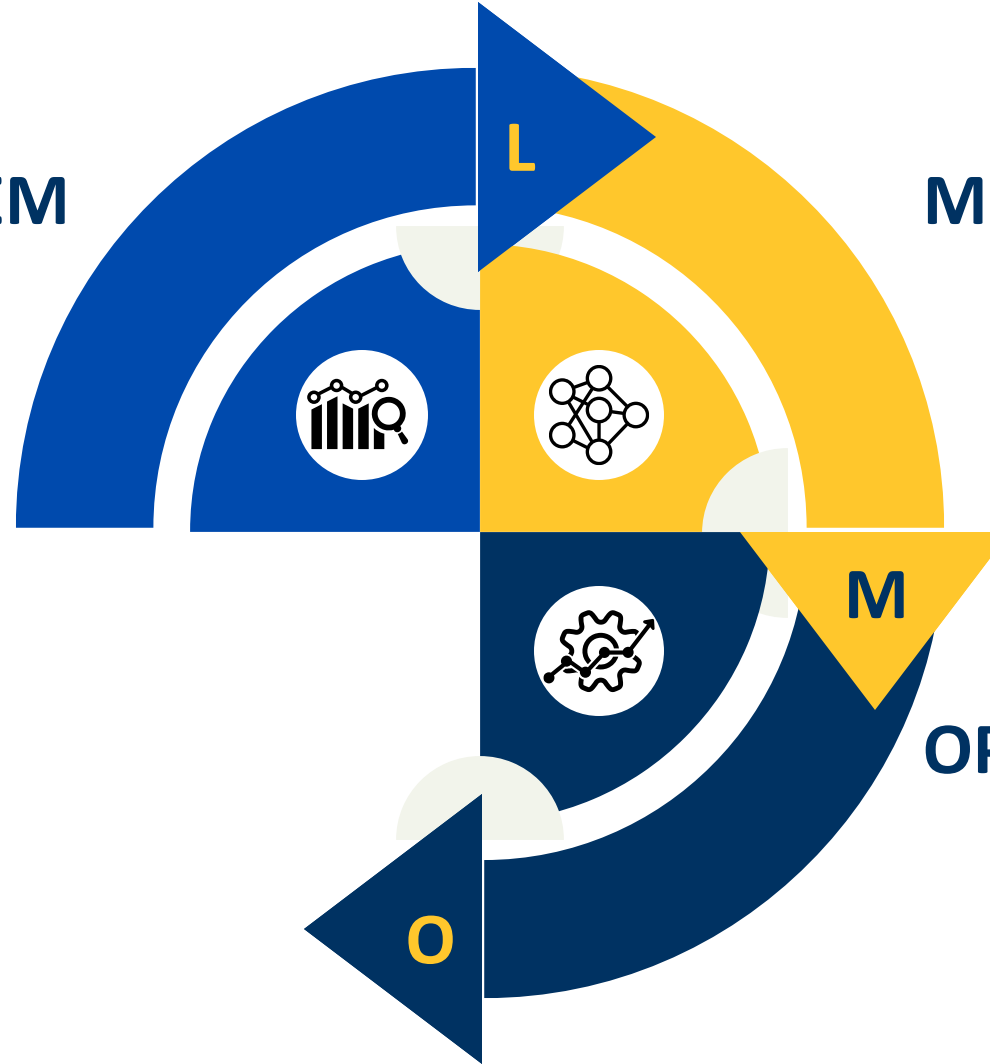
Optimization



6101346

LEARNING PROBLEM

Supervised learning
of scalar target values



MODEL DESIGN

Linear Regression
with basis functions
 $y(\vec{x}, \vec{w}) = \vec{x}^T \vec{w}$

OPTIMIZATION

Derive direct solution

$$E(w) = \frac{1}{2} \sum_{n=1}^N (t_n - y(\mathbf{x}_n, w))^2$$
$$w^* = \underset{w}{\operatorname{argmin}} E(w)$$



6101346

Error Function Minimization

- $E(\mathbf{w})$ is a quadratic function.

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N (\mathbf{t}_n - y(\mathbf{x}_n, \mathbf{w}))^2$$

Therefore, $\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}}$ is a linear function of $\mathbf{w}$ and hence solving $\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} = 0$ has one and only one solution. Let's now derive this solution:

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \left(\mathbf{t}_n - \mathbf{w}_0 - \sum_{j=1}^D \mathbf{w}_j \mathbf{x}_{nj} \right)^2 \xrightarrow{\text{Finding the optimum solution}} \frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} = 0$$

Error Function Minimization



6101346

$$\frac{\partial E(w)}{\partial w_0} = 0 \longrightarrow \frac{\partial \left(\frac{1}{2} \sum_{n=1}^N \left((t_n - w_0 - \sum_{d=1}^D w_d x_{nd}) \right)^2 \right)}{\partial w_0} = 0$$

Sum rule

$$\partial \sum f = \sum \partial f$$

Chain rule

$$\partial f^2 = 2f \partial f$$

Separating
the terms

$$\frac{\partial E(w)}{\partial w_0} = 0 \longrightarrow$$



6101346

Error Function Minimization

$$\frac{\partial E(w)}{\partial w_0} = 0 \longrightarrow \frac{\partial \left(\frac{1}{2} \sum_{n=1}^N \left((t_n - w_0 - \sum_{d=1}^D w_d x_{nd}) \right)^2 \right)}{\partial w_0} = 0$$

Sum rule

$$\frac{\partial \sum f}{\partial w_0} = \sum \frac{\partial f}{\partial w_0} = \frac{1}{2} \sum_{n=1}^N \frac{\partial \left((t_n - w_0 - \sum_{d=1}^D w_d x_{nd})^2 \right)}{\partial w_0}$$

Chain rule
 $\frac{\partial f^2}{\partial f} = 2f \frac{\partial f}{\partial w_0}$

$$= \frac{1}{2} \sum_{n=1}^N 2 \left(t_n - w_0 - \sum_{d=1}^D w_d x_{nd} \right) \left(\frac{\partial (t_n - w_0 - \sum_{d=1}^D w_d x_{nd})}{\partial w_0} \right)$$

Separating
the terms

$$= \sum_{n=1}^N \left(t_n - w_0 - \sum_{d=1}^D w_d x_{nd} \right) (-1) = - \left(\sum_{n=1}^N t_n - n w_0 - \sum_{n=1}^N \left(\sum_{d=1}^D w_d x_{nd} \right) \right)$$

$$\frac{\partial E(w)}{\partial w_0} = 0 \longrightarrow w_0 = \frac{1}{n} \left(\sum_{n=1}^N t_n - \sum_{n=1}^N \left(\sum_{d=1}^D w_d x_{nd} \right) \right)$$



6101346

Error Function Minimization

$$\frac{\partial E(\mathbf{w})}{\partial w_k} = 0 \quad \longrightarrow \quad \frac{\partial \left(\frac{1}{2} \sum_{n=1}^N \left((t_n - w_0 - \sum_{d=1}^D w_d x_{nd}) \right)^2 \right)}{\partial w_k} = 0$$

Sum rule

$$\frac{\partial \sum f}{\partial w_k} = \sum \frac{\partial f}{\partial w_k} = \frac{1}{2} \sum_{n=1}^N \frac{\partial \left((t_n - w_0 - \sum_{d=1}^D w_d x_{nd})^2 \right)}{\partial w_k}$$

Chain rule

$$\frac{\partial f^2}{\partial f} = 2f \frac{\partial f}{\partial w_k} \quad = \frac{1}{2} \sum_{n=1}^N 2 \left(t_n - w_0 - \sum_{d=1}^D w_d x_{nd} \right) \left(\frac{\partial (t_n - w_0 - \sum_{d=1}^D w_d x_{nd})}{\partial w_k} \right)$$

Reordering

$$\frac{\partial E(\mathbf{w})}{\partial w_k} = - \sum_{n=1}^N x_{nk} \left(t_n - w_0 - \sum_{d=1}^D w_d x_{nd} \right)$$



6101346

Error Function Minimization

$$\frac{\partial E(w)}{\partial w_k} = - \sum_{n=1}^N x_{nk} \left(t_n - w_0 - \sum_{d=1}^D w_d x_{nd} \right)$$

$$\sum_{n=1}^N x_{nk} t_n = \sum_{n=1}^N x_{nk} \left(w_0 + \sum_{d=1}^D w_d x_{nd} \right) \rightarrow \underbrace{\sum_{n=1}^N x_{nk} t_n}_{\mathbb{X}^T T} = \underbrace{w_0 \sum_{n=1}^N x_{nk} + \sum_{d=1}^D w_d \sum_{n=1}^N x_{nk} x_{nd}}_{\mathbb{X}^T \mathbb{X} w}$$

Normal equations for the least squares problem

$$\mathbb{X}^T T = \mathbb{X}^T \mathbb{X} w$$

Multiplying both sides by $(\mathbb{X}^T \mathbb{X})^{-1}$

$$(\mathbb{X}^T \mathbb{X})^{-1} \mathbb{X}^T T = w$$

Takeaway $\rightarrow w^* = (\mathbb{X}^T \mathbb{X})^{-1} \mathbb{X}^T T$

Assuming $\mathbb{X}^T \mathbb{X}$ is invertible

Geometric Interpretation

- Error Minimization
- Geometric Interpretation
- Evaluation
- Regularized Least Squares
- When Normal Equation Gets Tricky



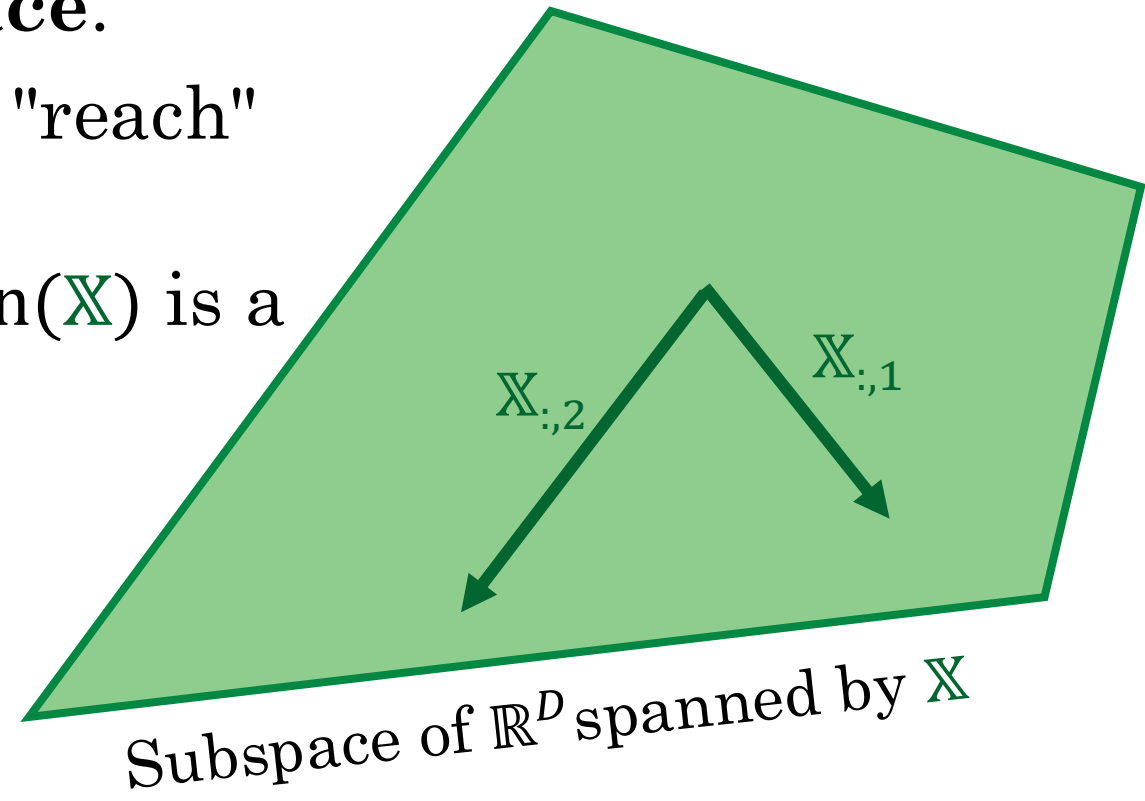
6101346



6101346

[Linear Algebra] Span

- The set of all possible linear combinations of the columns of $\mathbb{X}$ is called the **span** of $\mathbb{X}$ (denoted $\text{span}(\mathbb{X})$), also called the **column space**.
- Intuitively, this is all vectors you can "reach" using the columns of $\mathbb{X}$.
- If each column of $\mathbb{X}$ has length D , $\text{span}(\mathbb{X})$ is a subspace of $\mathbb{R}^D$.



[Linear Algebra] Matrix-Vector Multiplication



6101346

- **Approach 1:** So far, we've thought of our model as horizontally stacked predictions per datapoint:

$$\begin{bmatrix} | \\ \mathbb{Y} \\ | \end{bmatrix} = \begin{bmatrix} \text{---} & \mathbf{x}_1^T & \text{---} \\ \text{---} & \mathbf{x}_2^T & \text{---} \\ & \vdots & \\ \text{---} & \mathbf{x}_N^T & \text{---} \end{bmatrix} \begin{bmatrix} \text{---} \\ \text{---} \\ \text{---} \\ \text{---} \end{bmatrix} \begin{matrix} \\ \\ w \\ \end{matrix}$$

Matrix sizes
 $N \times 1 = (N \times (D + 1)) ((D + 1) \times 1)$

- **Approach 2:** However, it is helpful sometimes to think of matrix-vector multiplication as performed by columns. We can also think of $\mathbb{Y}$ as a **linear combination of feature vectors**, scaled by **parameters**.

$$\begin{bmatrix} | \\ \mathbb{Y} \\ | \end{bmatrix} = \begin{bmatrix} | & & | \\ \mathbb{X}_{:,0} & \dots & \mathbb{X}_{:,D} \\ | & & | \end{bmatrix} \begin{bmatrix} \text{---} \\ \text{---} \\ \text{---} \\ \text{---} \end{bmatrix} \begin{matrix} \\ \\ w \\ \end{matrix} = w_0 \begin{bmatrix} | \\ \mathbb{X}_{:,0} \\ | \end{bmatrix} + w_1 \begin{bmatrix} | \\ \mathbb{X}_{:,1} \\ | \end{bmatrix} + \dots + w_D \begin{bmatrix} | \\ \mathbb{X}_{:,D} \\ | \end{bmatrix}$$

Prediction Is a Linear Combination of Columns

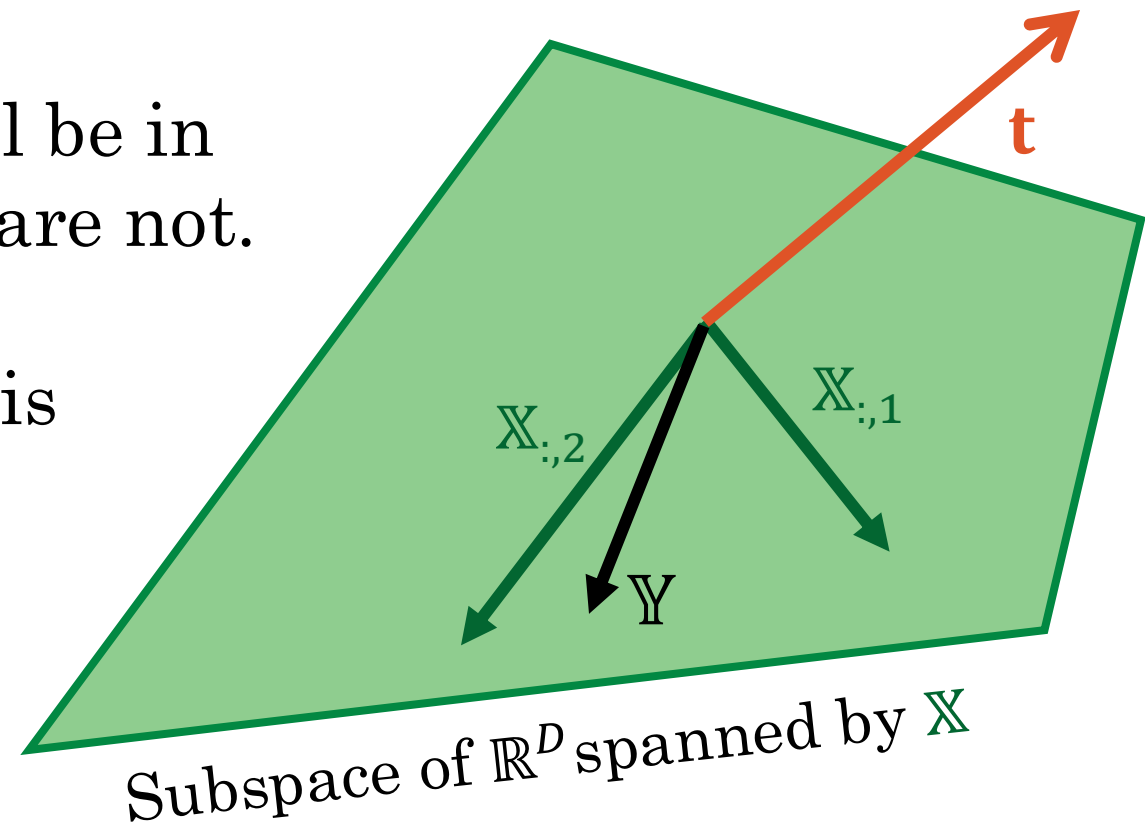


6101346

Our prediction of $\mathbb{Y}(\mathbb{X}, \vec{w}) = \mathbb{X} \vec{w}$ is a linear combination of columns of $\mathbb{X}$.

Interpret: Our linear prediction $\mathbb{Y}$ will be in $\text{span}(\mathbb{X})$, even if ground-truth values $\mathbf{t}$ are not.

Goal: Find vector of $\mathbb{Y}$ in $\text{span}(\mathbb{X})$ that is closest to $\mathbf{t}$.





**What's the geometry word for
'closest point in a subspace'?**

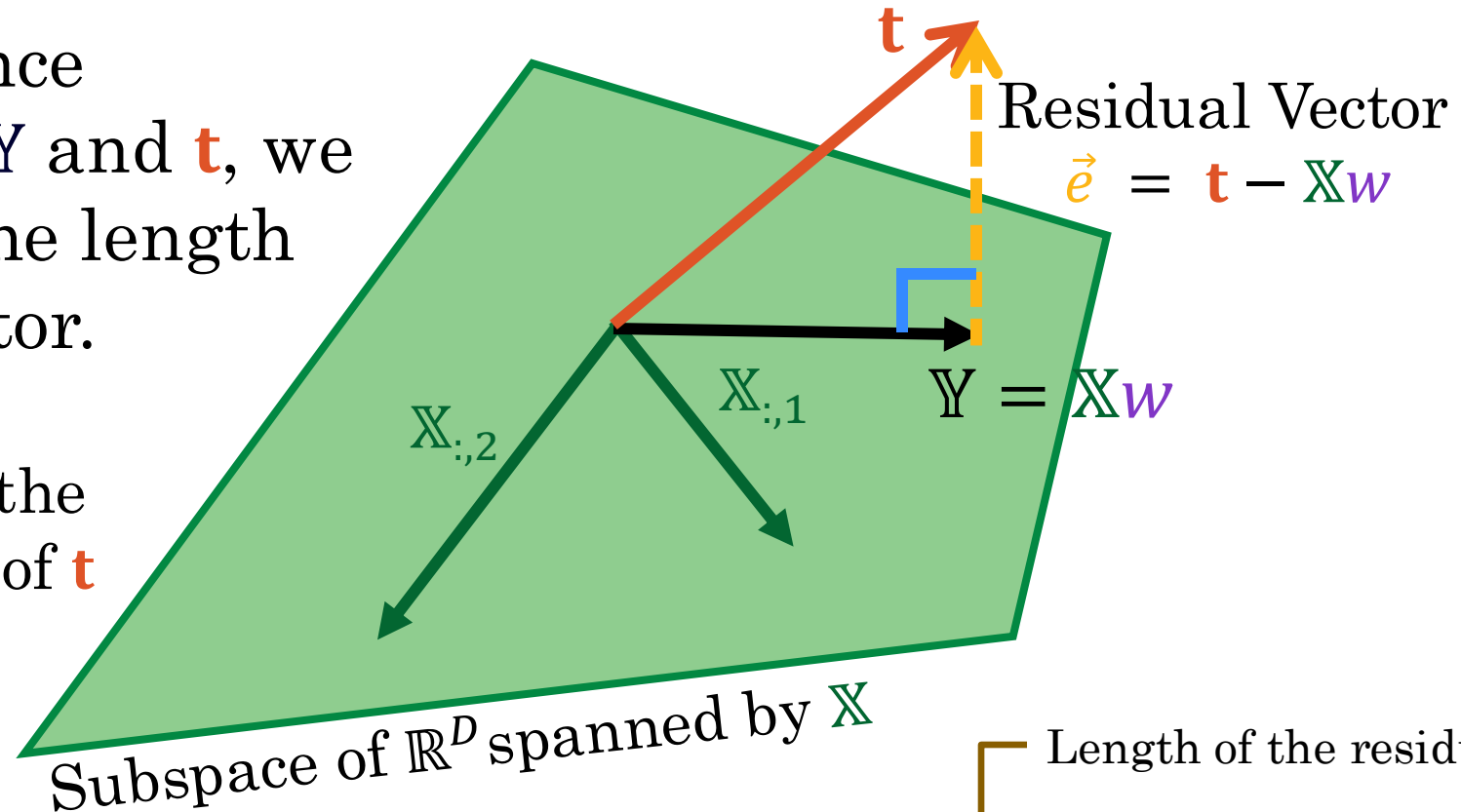


6101346

Finding Optimum Predictions ($\mathbb{Y}$)

To minimize distance between vector of $\mathbb{Y}$ and $\mathbf{t}$, we should minimize the length of the residual vector.

$\vec{e}$ is minimized if it is the orthogonal projection of $\mathbf{t}$ on the $\text{span}(\mathbb{X})$.

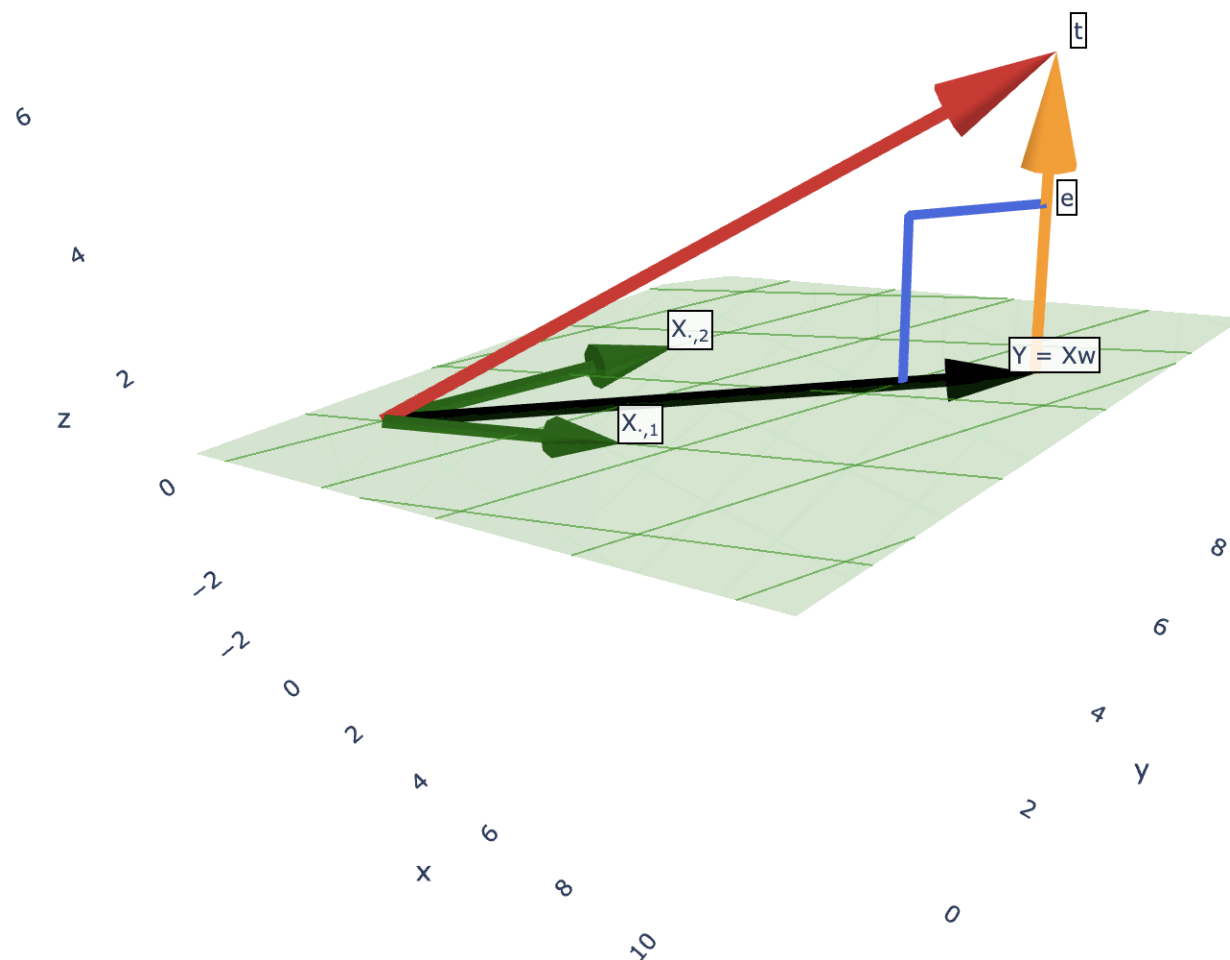


Length of the residual vector

$$\begin{aligned} L_2 - \text{norm}(\vec{e}) &= \|\vec{e}\|_2 \\ &= \sqrt{\sum_{d=0}^D e_d^2} \end{aligned}$$

Geometry of Least Squares in Plotly

[Interactive link](#)

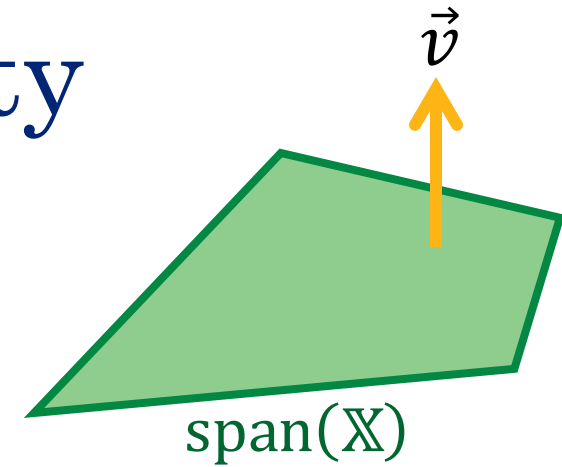


[Linear Algebra] Orthogonality



6101346

- A vector $\vec{v}$ is orthogonal to the $\text{span}(\mathbb{X})$ if and only if it is orthogonal to all columns of $\mathbb{X}$.



Assuming $\vec{v} \in \mathbb{R}^N$ and $\mathbb{X} = \begin{bmatrix} | & & | \\ \mathbb{X}_{:,0} & \dots & \mathbb{X}_{:,D} \\ | & & | \end{bmatrix} \in \mathbb{R}^{N \times D}$

$\vec{v}$ orthogonal to each column means:

$$\mathbb{X}_{:,0}^T \vec{v} = 0$$

$$\mathbb{X}_{:,1}^T \vec{v} = 0$$

$$\vdots$$

$$\mathbb{X}_{:,D}^T \vec{v} = 0$$

$$\Rightarrow \begin{bmatrix} \mathbb{X}_{:,0}^T \vec{v} \\ \mathbb{X}_{:,1}^T \vec{v} \\ \vdots \\ \mathbb{X}_{:,D}^T \vec{v} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

$$\Rightarrow \mathbb{X}^T \vec{v} = 0$$

We will use this shortly



6101346

Going Back to Our Error Function

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N (\mathbf{t}_n - y(\mathbf{x}_n, \mathbf{w}))^2 = \frac{1}{2} (\|\vec{e}\|_2)^2$$

Equivalently, vector $\vec{e}$ is orthogonal to the $\text{span}(\mathbb{X})$, meaning that:

$$\mathbb{X}^T \vec{e} = 0 \quad \text{Adding the definition of residual}$$

$$\mathbb{X}^T (\mathbf{t} - \mathbb{Y}(\mathbb{X}, \mathbf{w}^*)) = 0 \quad \text{Adding formula for } \mathbb{Y}(\mathbb{X}, \mathbf{w}^*) = \mathbb{X}\mathbf{w}^*$$

$$\mathbb{X}^T \mathbf{t} - \mathbb{X}^T \mathbb{X} \mathbf{w}^* = 0 \quad \text{Moving terms}$$

$$\mathbb{X}^T \mathbf{t} = \mathbb{X}^T \mathbb{X} \mathbf{w}^*$$

Normal
Equation

If $\mathbb{X}^T \mathbb{X}$ is
invertible

$$\mathbf{w}^* = (\mathbb{X}^T \mathbb{X})^{-1} \mathbb{X}^T \mathbf{t}$$

Evaluation

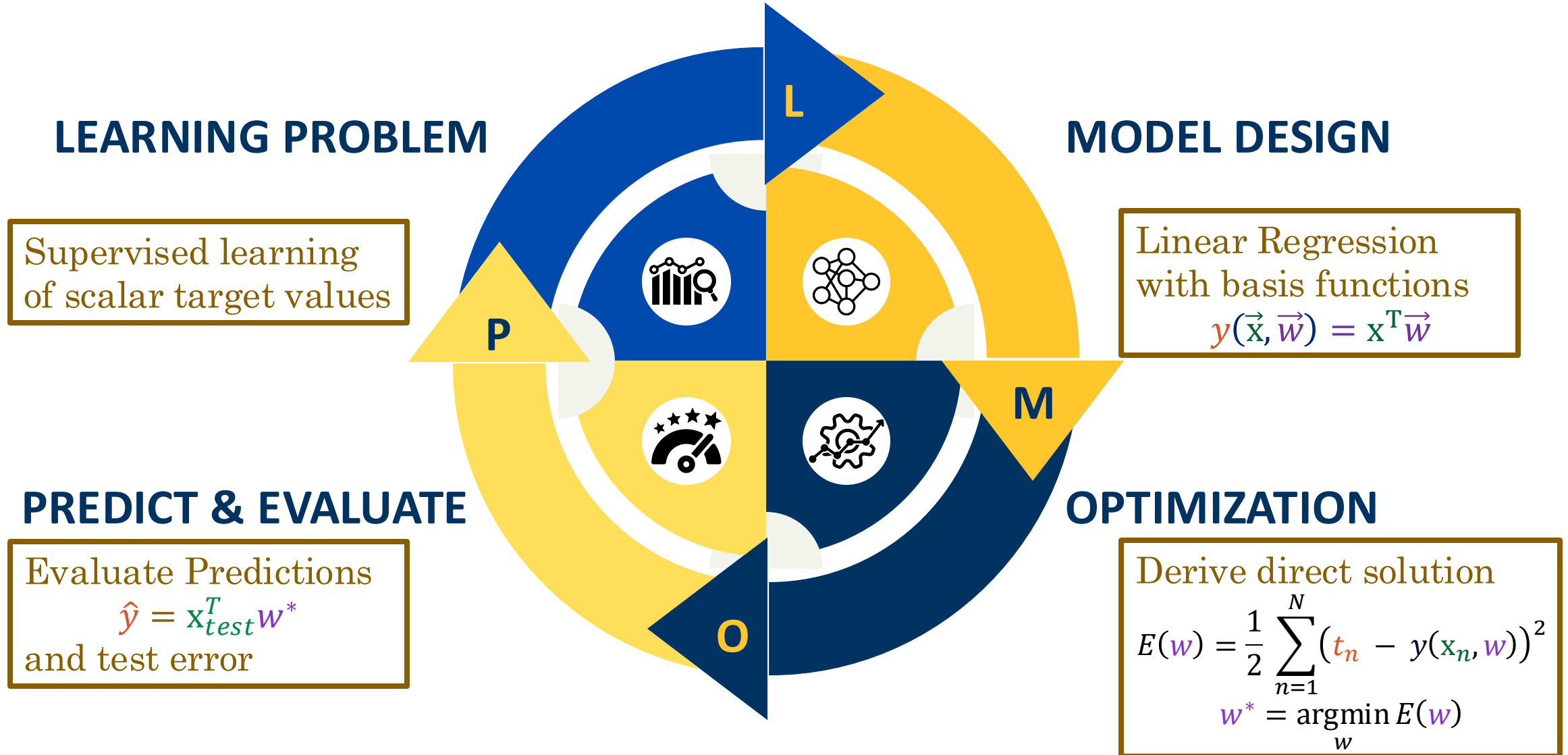
- Error Minimization
- Geometric Interpretation
- Evaluation
- Regularized Least Squares
- When Normal Equation Gets Tricky



Predict and Evaluate



6101346



Evaluation - Visualization

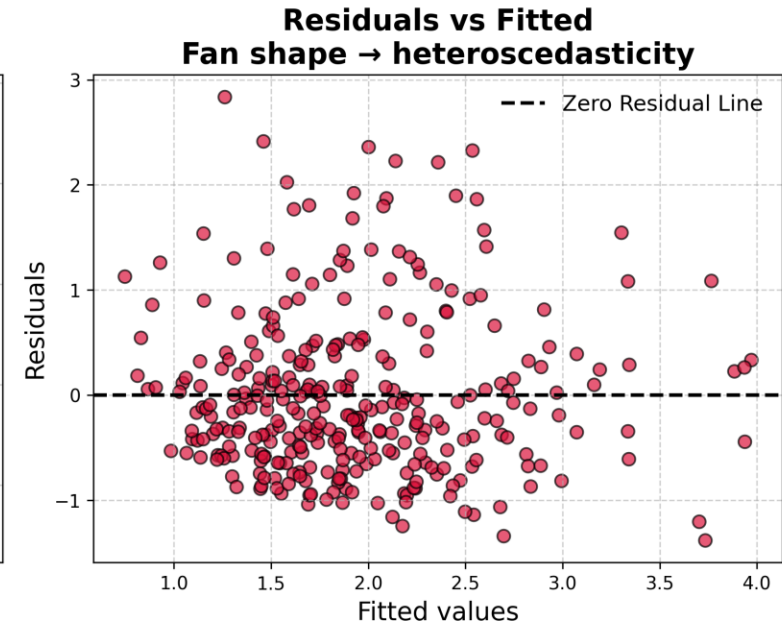
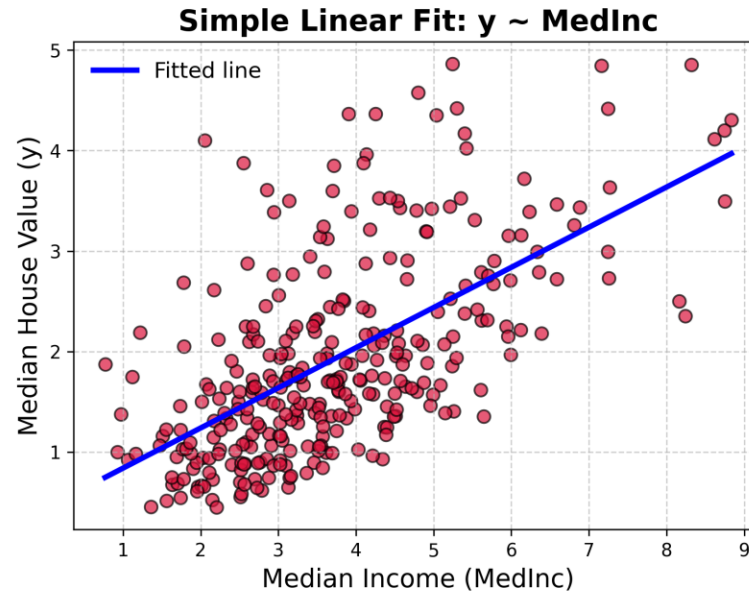
Residual plot shows the trend of the residuals $e_n = (t_n - y(x_n, w))$ with respect to predictions $y(x_n, w)$.

What a good residual plot looks like:

- Points scattered randomly around 0 (no pattern).

- Roughly constant vertical spread (homoscedasticity).

- No obvious trend with fitted values or with predictors.





When you see a fan shape in the residual plot, what comes to mind?

Evaluation - Metrics



6101346

Evaluation - Metrics



6101346

Mean Squared Error (MSE)

$$\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)$$

Evaluation - Metrics



6101346

Mean Squared Error (MSE)

$$\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)$$

Root Mean Squared Error
(RMSE)

$$\sqrt{\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)}$$

Moves the metric
back to the original
unit of the data
compared to MSE

Evaluation - Metrics



6101346

Mean Squared Error (MSE)

$$\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)$$

Root Mean Squared Error
(RMSE)

$$\sqrt{\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)}$$

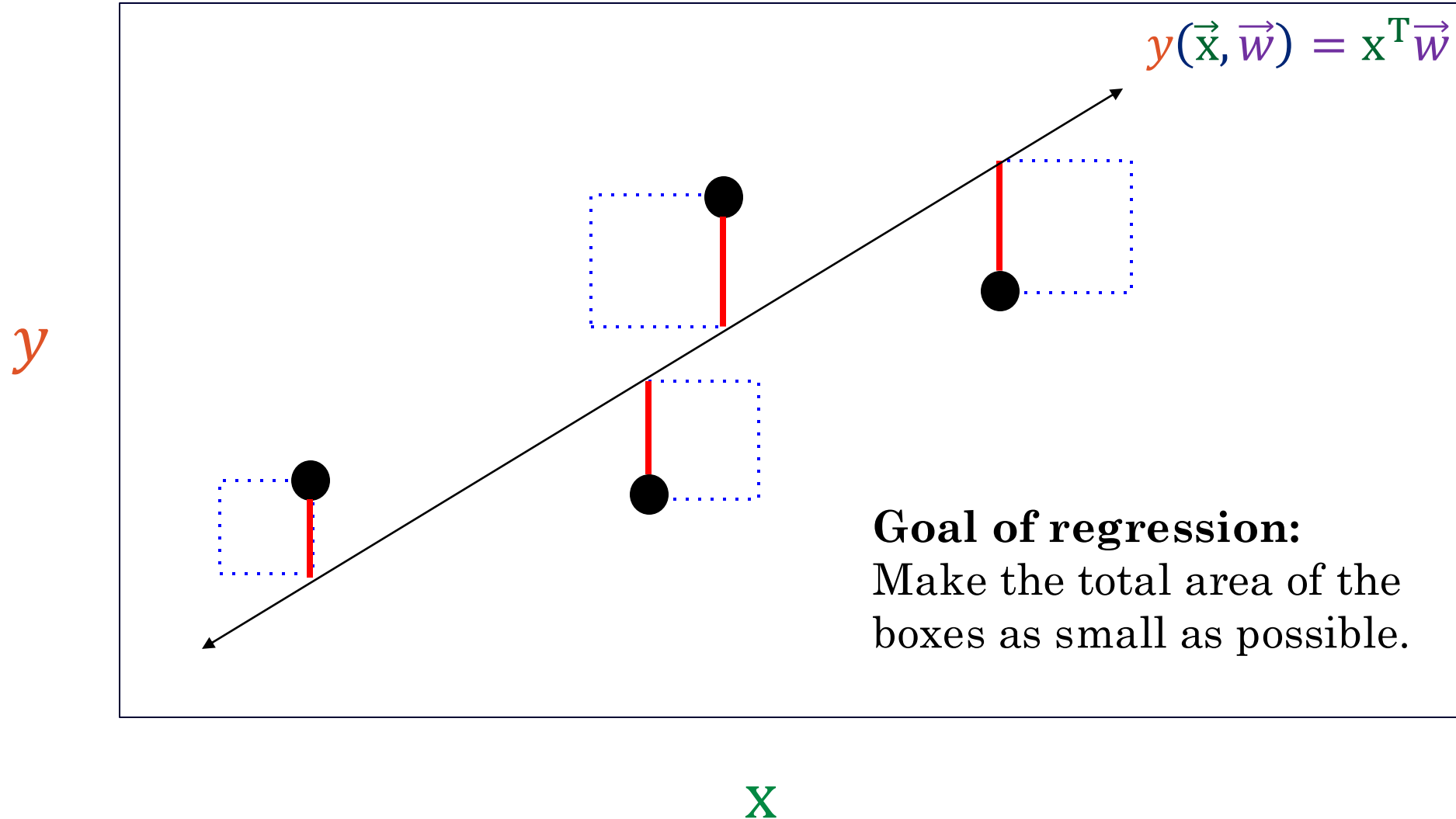
R-Squared (R^2) Score

$$1 - \frac{\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2}{\sum_{n=1}^N (t_n - \bar{t})^2}$$

Visualizing the Sum of Squared Error of Regression Model



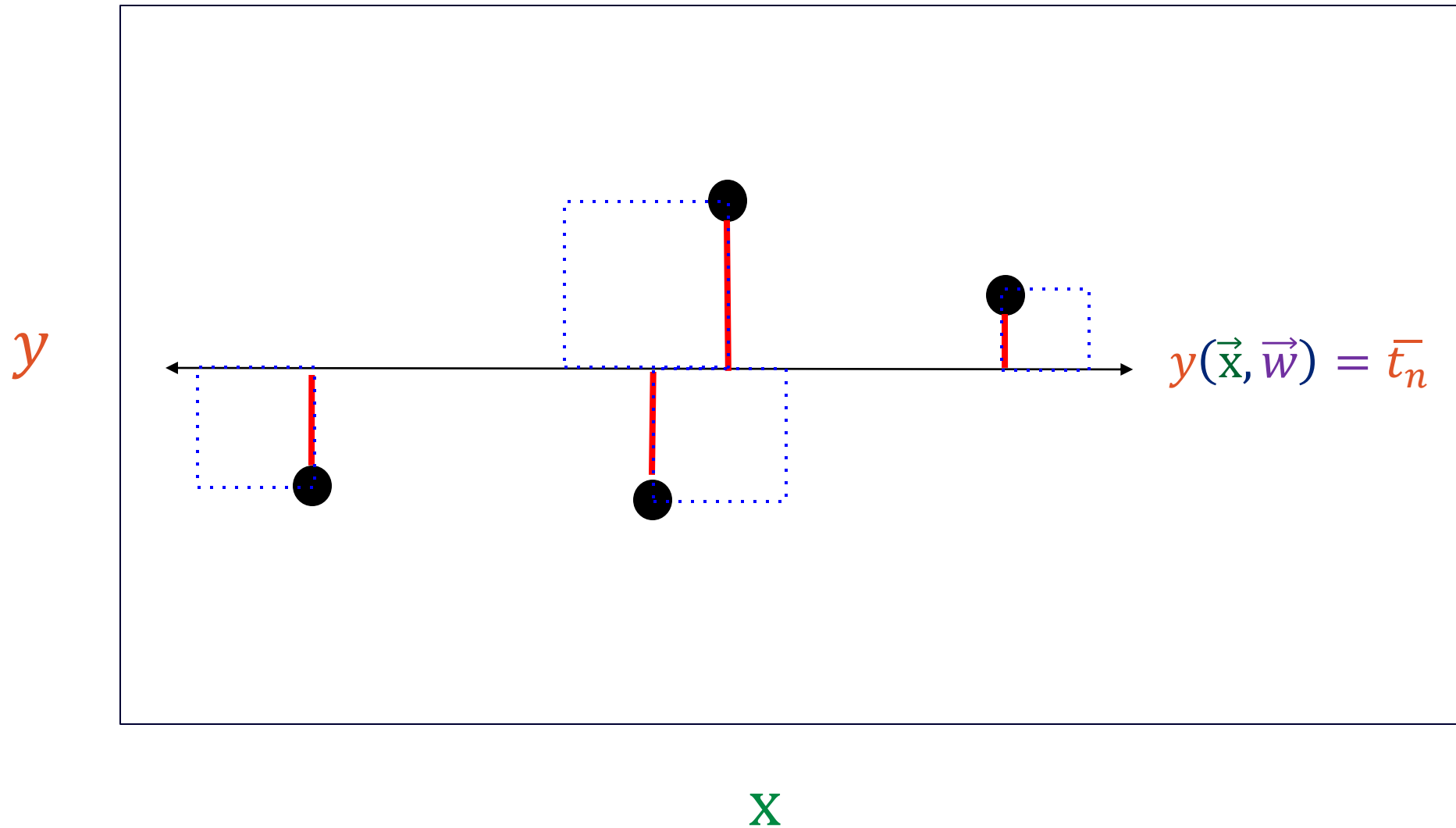
6101346



Visualizing the Sum of Squared Error of Intercept Model



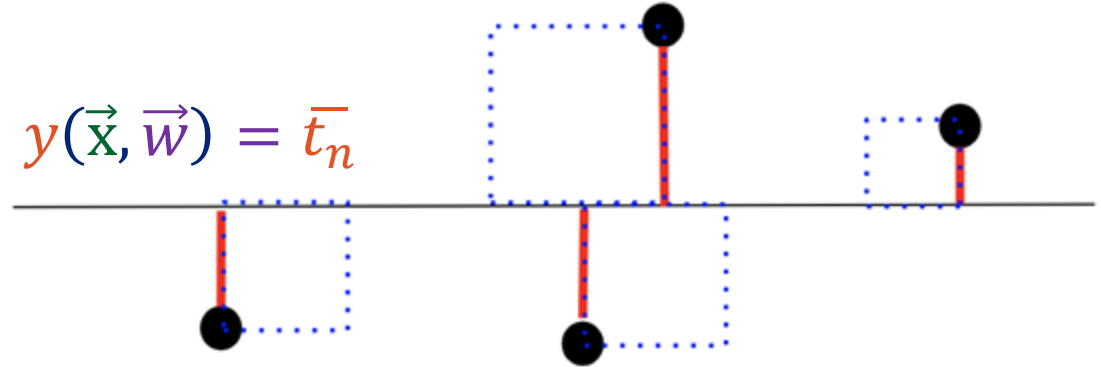
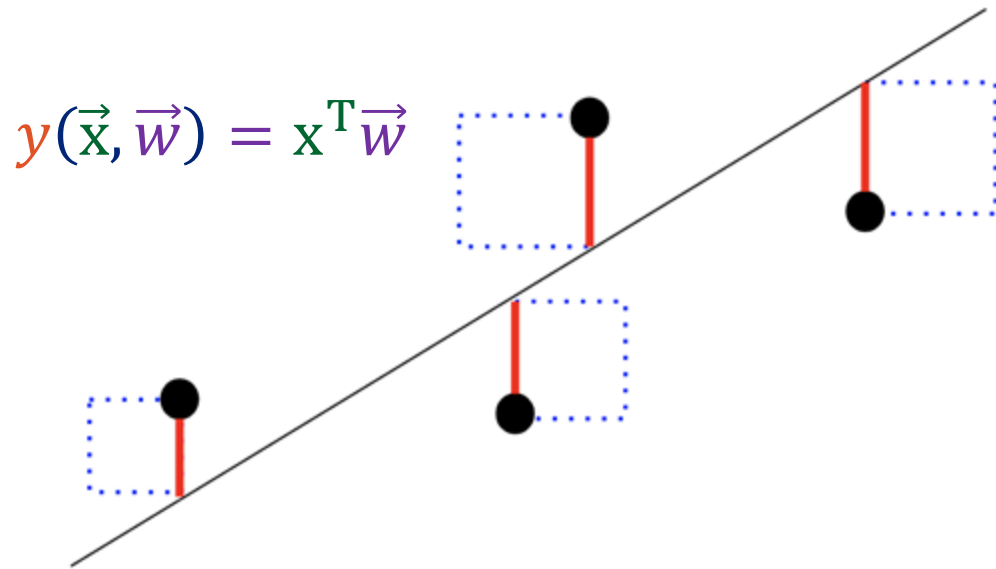
6101346



R²: Quality of the Fit Relative to Intercept Model



6101346



$$R^2 = \frac{\Delta \text{ in area}}{\text{Constant Model Area}} = 1 - \frac{\sum_{n=1}^N (t_n - y(\vec{x}_n, \vec{w}))^2}{\sum_{n=1}^N (t_n - \bar{t}_n)^2}$$

unitless and only compares performance relative to mean baseline



6101346

Evaluation - Metrics

Mean Squared Error (MSE)

$$\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)$$

Root Mean Squared Error (RMSE)

$$\sqrt{\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)}$$

R-Squared (R^2) Score

$$1 - \frac{\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2}{\sum_{n=1}^N (t_n - \bar{t})^2}$$

Mean Absolute Error (MAE)

$$\frac{1}{N} \left(\sum_{n=1}^N |t_n - y(\mathbf{x}_n, \mathbf{w})| \right)$$

In the same unit as the data; similar to MSE but differs in how the penalization applies.



6101346

Evaluation - Metrics

Mean Squared Error (MSE)

$$\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)$$

Root Mean Squared Error (RMSE)

$$\sqrt{\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)}$$

R-Squared (R^2) Score

$$1 - \frac{\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2}{\sum_{n=1}^N (t_n - \bar{t})^2}$$

Mean Absolute Error (MAE)

$$\frac{1}{N} \left(\sum_{n=1}^N |t_n - y(\mathbf{x}_n, \mathbf{w})| \right)$$

Mean Absolute Percentage Error (MAPE)

$$\frac{1}{N} \left(\sum_{n=1}^N \frac{t_n - y(\mathbf{x}_n, \mathbf{w})}{t_n} \right) \times 100$$

Regularized Least Squares

- Error Minimization
- Geometric Interpretation
- Evaluation
- Regularized Least Squares
- When Normal Equation Gets Tricky

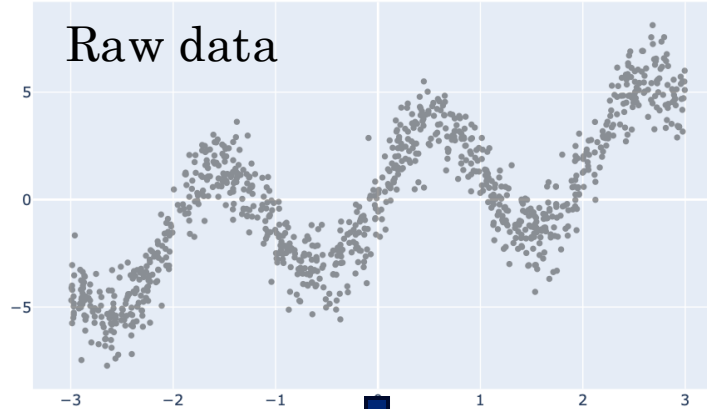


6101346

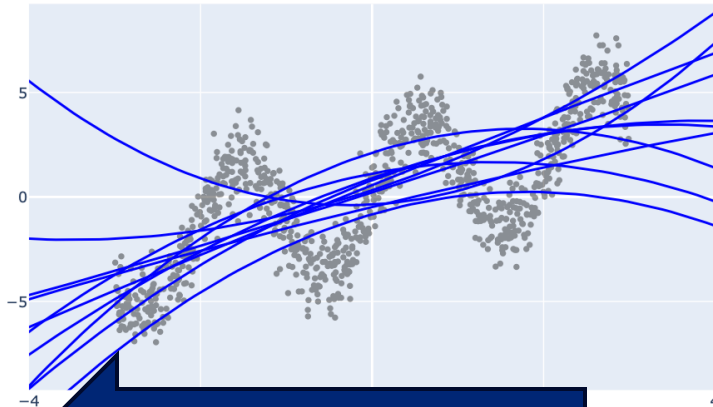
Complexity and Overfitting



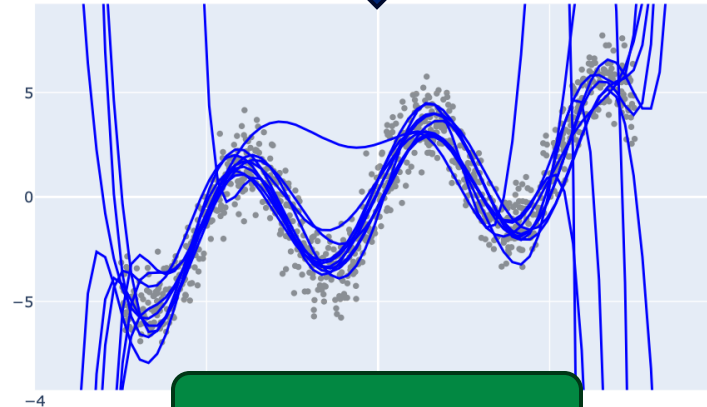
6101346



Fit models to
samples of data

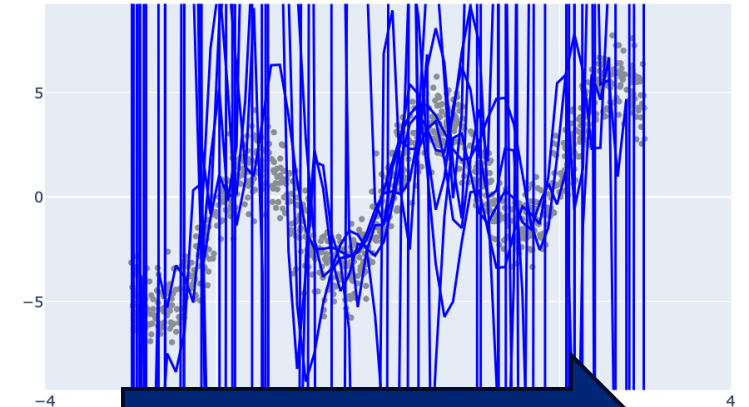


Underfitting



“Sweet Spot”

More complex isn't
always better.



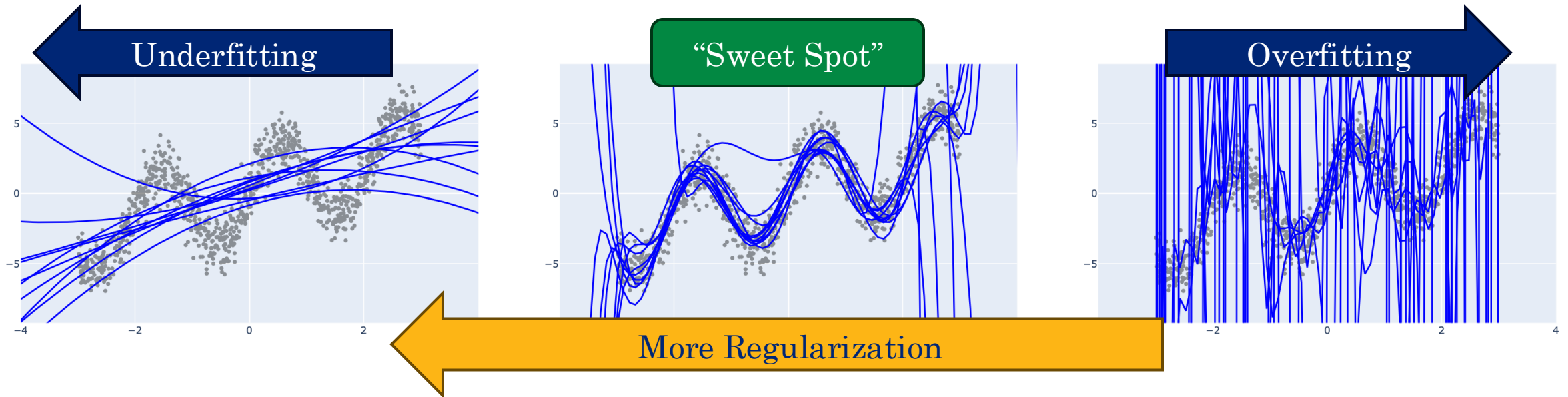
Overfitting

Regularization



6101346

Regularization is the process of adding **constraints** or **penalties** to the learning process to **improve generalization**.



Many **models** and **learning algorithms** have methods to **tune** the **regularization** during the training process.



6101346

Regularization

Regularization restricts complexity by limiting the magnitudes of the model parameters w .

We capture this restriction through the function $\text{Reg}[w]$ and restricting its value to be less than C .

Our Error function becomes a constrained function, so we minimize:

$$\text{Error}[h_w; \mathcal{D}_{\text{training}}] \quad \text{such that} \quad \text{Reg}[w] < C$$



6101346

Regularization – Lagrangian Duality

Our Error function becomes a constrained function, so we minimize:

$$\text{Error}[h_w; \mathcal{D}_{\text{training}}] \quad \text{such that} \quad \text{Reg}[w] < C$$

This constraint complicates our optimization.

What if we assign a penalty for each violation of the constraint?

Idea of Lagrangian

Introduce a penalty for breaking the constraint.

$$\underbrace{\lambda}_{\text{Imposed penalty}} \underbrace{(\text{Reg}[w] - C)}_{\text{Violation magnitude}}$$

The updated Error function

$$E(w) = \text{Error}[h_w; \mathcal{D}_{\text{training}}] + \lambda \text{Reg}[w]$$

Note: we did not include $-\lambda C$ since it is a constant and won't impact the optimization



6101346

From **Reg**[w] $\rightarrow$ Norms

- One simple example of **Reg**[w] is the sum of the squares of the weight vector:

$$\frac{1}{2} \sum w_d^2 = \frac{1}{2} w^T w$$

- Generally, **norms** are the standard choices for **Reg**[w].
- What is a norm?
 - **Nonnegativity & definiteness:** $\|v\| \geq 0$, $\|v\| = 0 \iff \vec{v} = \vec{0}$
 - **Positive homogeneity:** $\|\alpha v\| = |\alpha| \|v\|$
 - **Triangle inequality:** $\|u + v\| \leq \|u\| + \|v\|$
- Norm definition: $L_p(w)$: $\|w\|_p = (\sum_d |w_d|^p)^{\frac{1}{p}}$



6101346

From **Reg**[*w*] → Norms

- Norm definition: $L_p(w): \|w\|_p = (\sum_d |w_d|^p)^{\frac{1}{p}}$
- **Example of norms:**
 - $L_1(w): \|w\|_1 = \sum_d |w_d|$
 - $L_2(w): \|w\|_2 = \sqrt{\sum_d w_d^2}$
 - $L_\infty(w): \|w\|_\infty = \max_d |w_d|$



6101346

Reg[w] $\rightarrow$ Norms $\rightarrow L_1$ Regularization

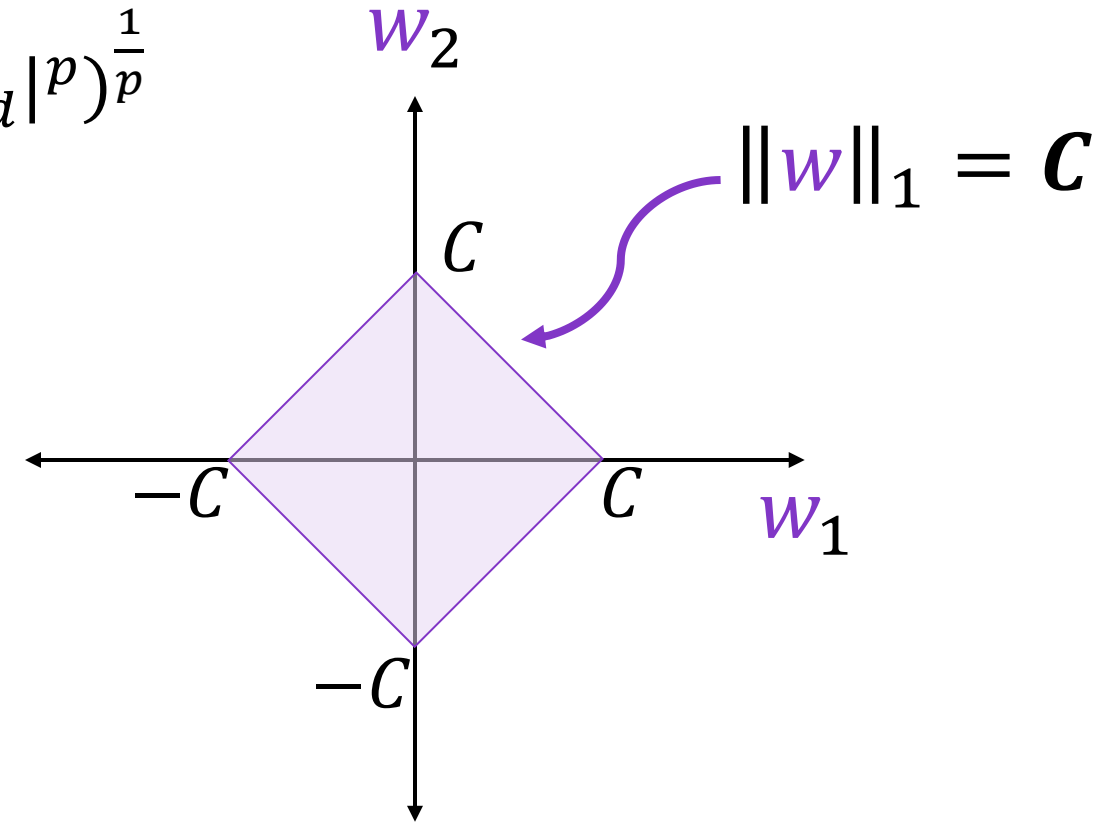
- Norm definition: $L_p(w): \|w\|_p = (\sum_d |w_d|^p)^{\frac{1}{p}}$

- Example of norms:

- $L_1(w): \|w\|_1 = \sum_d |w_d|$

- $L_2(w): \|w\|_2 = \sqrt{\sum_d w_d^2}$

- $L_\infty(w): \|w\|_\infty = \max_d |w_d|$





6101346

Reg[w] $\rightarrow$ Norms $\rightarrow L_2$ Regularization

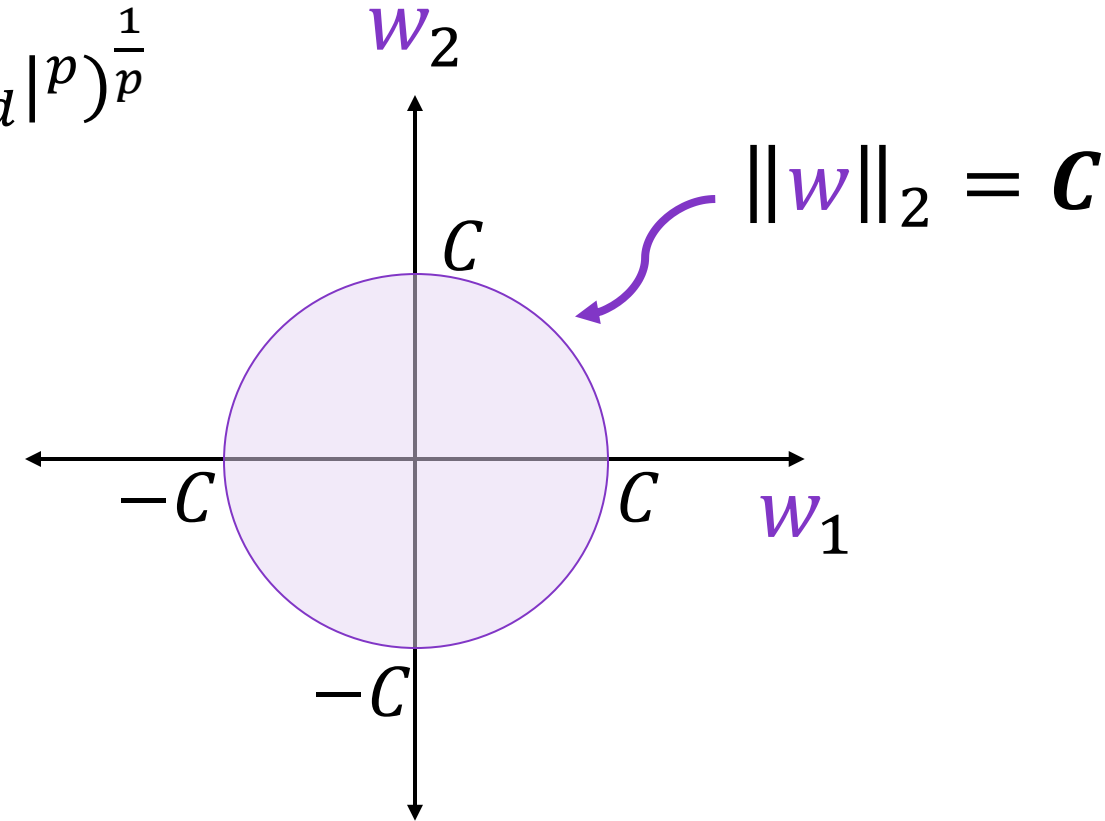
- Norm definition: $L_p(w): \|w\|_p = (\sum_d |w_d|^p)^{\frac{1}{p}}$

- Example of norms:

- $L_1(w): \|w\|_1 = \sum_d |w_d|$

- $L_2(w): \|w\|_2 = \sqrt{\sum_d w_d^2}$

- $L_\infty(w): \|w\|_\infty = \max_d |w_d|$





6101346

Regularization

We typically use **iterative optimization algorithms** to train (fit) the model by solving the following problem:

$$w^* = \arg \min_w \left(\underbrace{\mathbf{E}[w; \mathcal{D}_{\text{training}}]}_{\text{Data-dependent Error}} + \lambda \mathbf{Reg}[w] \right)$$

Regularization



We typically use **iterative optimization algorithms** to train (fit) the model by solving the following problem:

$$w^* = \arg \min_w \left(\underbrace{E_D(w)}_{\text{Data-dependent Error}} + \lambda \text{Reg}[w] \right)$$

Data-dependent
Error

Regularization



6101346

We typically use **iterative optimization algorithms** to train (fit) the model by solving the following problem:

$$w^* = \arg \min_w \left(\underbrace{E_D(w)}_{\text{Data-dependent Error}} + \lambda \underbrace{E_w[w]}_{\text{Regularization Function}} \right)$$

Data-dependent Error Regularization Function

Regularization



6101346

We typically use **iterative optimization algorithms** to train (fit) the model by solving the following problem:

$$w^* = \arg \min_w (\underbrace{E_D(w)}_{\text{Data-dependent Error}} + \underbrace{\lambda E_w[w]}_{\text{Regularizer Hyperparameter}})$$

Data-dependent
Error

Regularizer
Hyperparameter

Regularization
Function

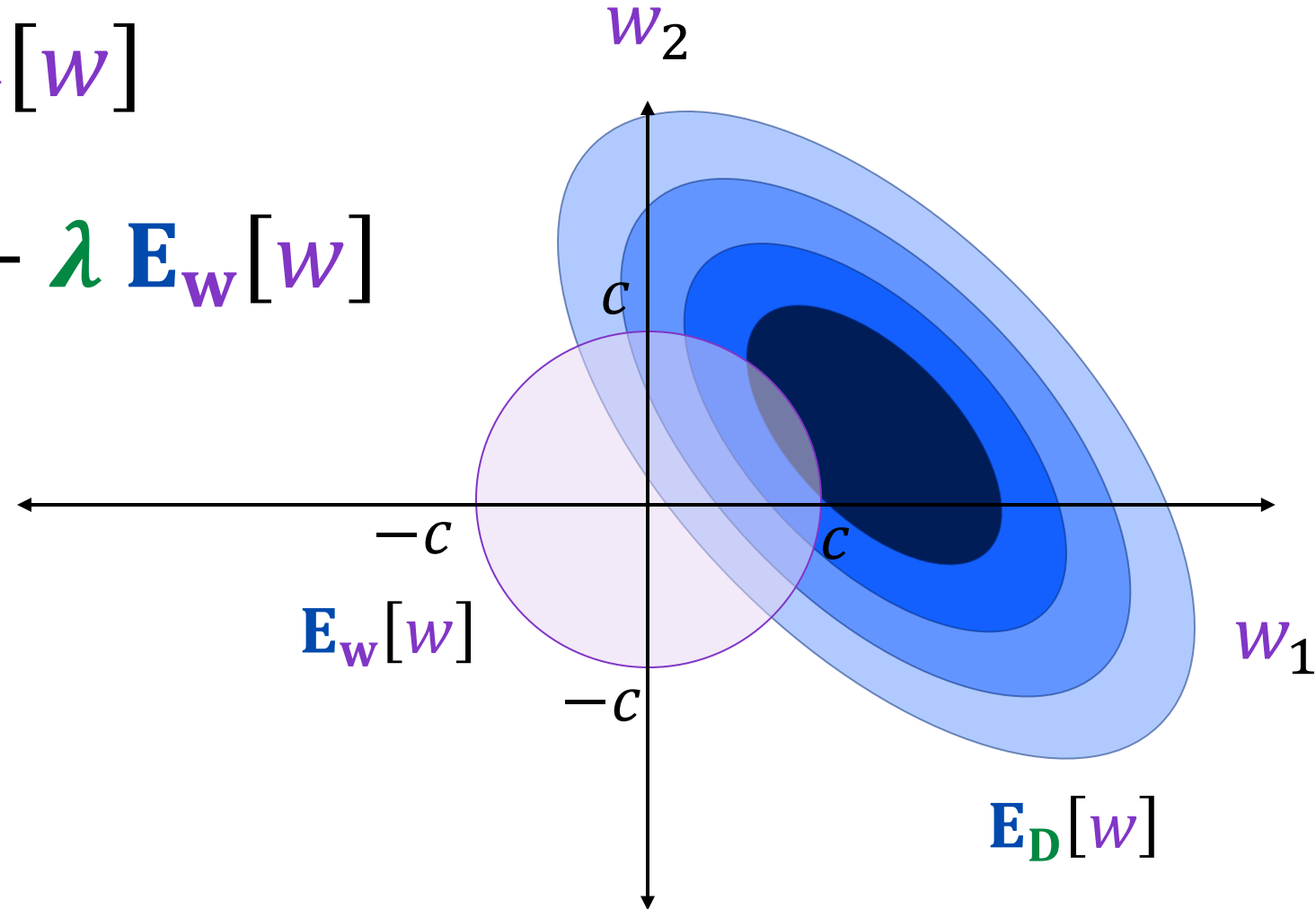
Linear Regression with L2 Regularization (Ridge)



6101346

$$E(w) = E_D[w] + \lambda E_w[w]$$

$$w^* = \arg \min_w E_D[w] + \lambda E_w[w]$$



L2 Regularization (Ridge)

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \underbrace{\mathbf{E}_{\mathbf{D}}[\mathbf{w}] + \lambda \mathbf{E}_{\mathbf{w}}[\mathbf{w}]}_{\text{L2 Regularization (Ridge)}}$$

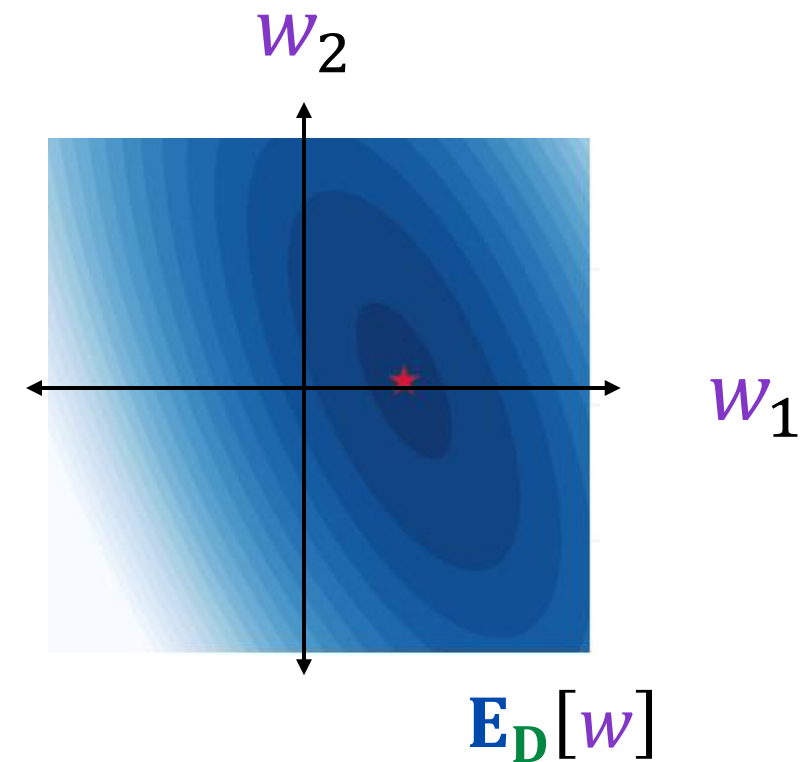
$$\frac{1}{2} \sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2$$

$$\frac{1}{2} (\|t_n - y(\mathbf{x}_n, \mathbf{w})\|_2)^2$$

$$\frac{1}{2} (\|e_n\|_2)^2$$

length of the
residual vector

$$L_2[\mathbf{w}] = \sqrt{\sum_d w_d^2}$$



6101346

L2 Regularization (Ridge)



$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \underbrace{\mathbf{E}_{\mathbf{D}}[\mathbf{w}]} + \underbrace{\lambda \mathbf{E}_{\mathbf{w}}[\mathbf{w}]}$$

$$\frac{1}{2} \sum_{n=1}^N (\mathbf{t}_n - y(\mathbf{x}_n, \mathbf{w}))^2$$

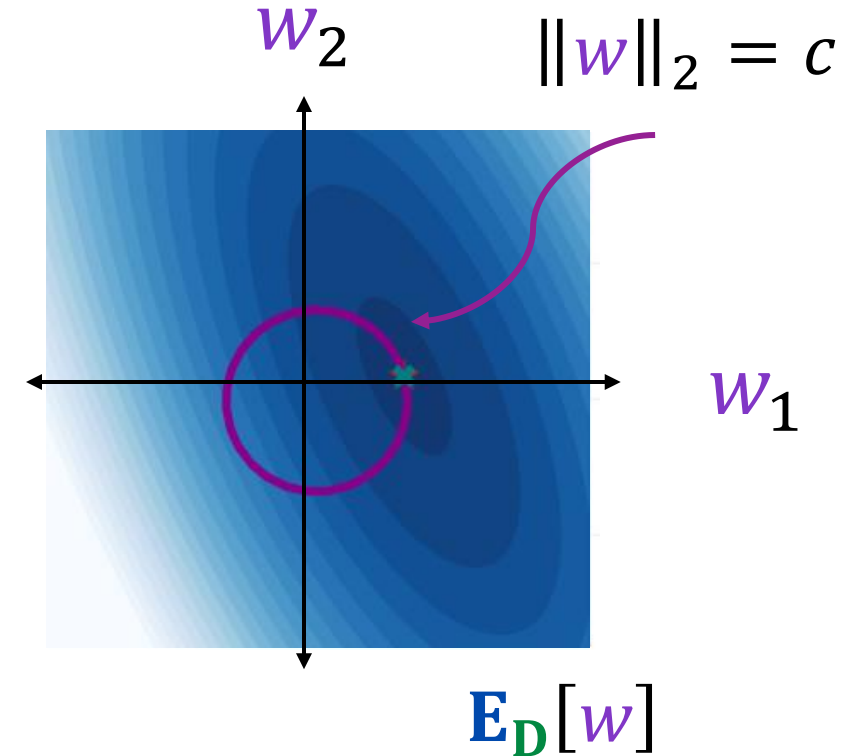
$$\frac{1}{2} L_2[\mathbf{w}]^2 = \frac{1}{2} \sum_d \mathbf{w}_d^2$$

$$\frac{1}{2} (\|\mathbf{t}_n - y(\mathbf{x}_n, \mathbf{w})\|_2)^2$$

$$\frac{1}{2} (\|e_n\|_2)^2$$

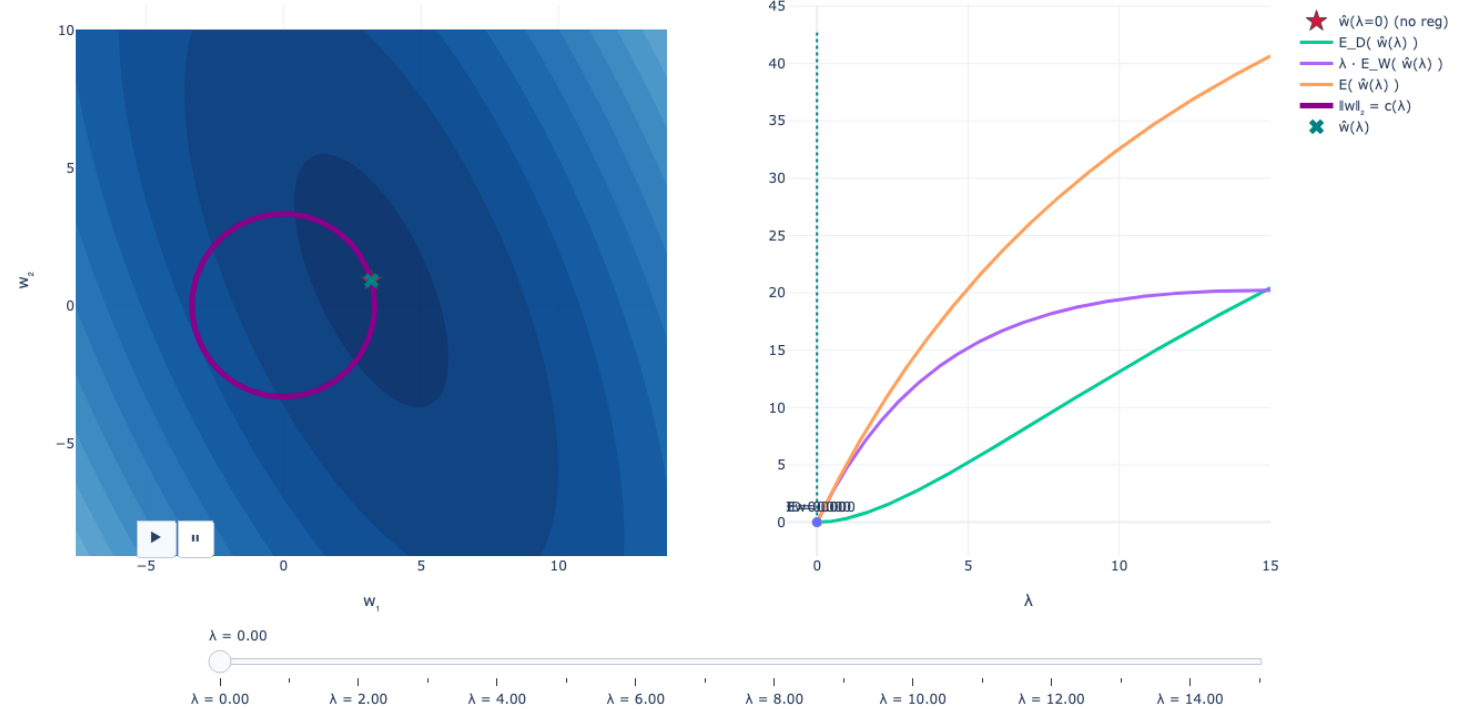
length of the
residual vector

$$L_2[\mathbf{w}] = \sqrt{\sum_d \mathbf{w}_d^2}$$



L2 Regularization Demo

[Interactive link](#)



L2 Regularization (Lasso)

$$L_1[\mathbf{w}] = \sum_j |\mathbf{w}_j|$$



$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \underbrace{\mathbf{E}_{\mathbf{D}}[\mathbf{w}]} + \underbrace{\lambda \mathbf{E}_{\mathbf{w}}[\mathbf{w}]}$$

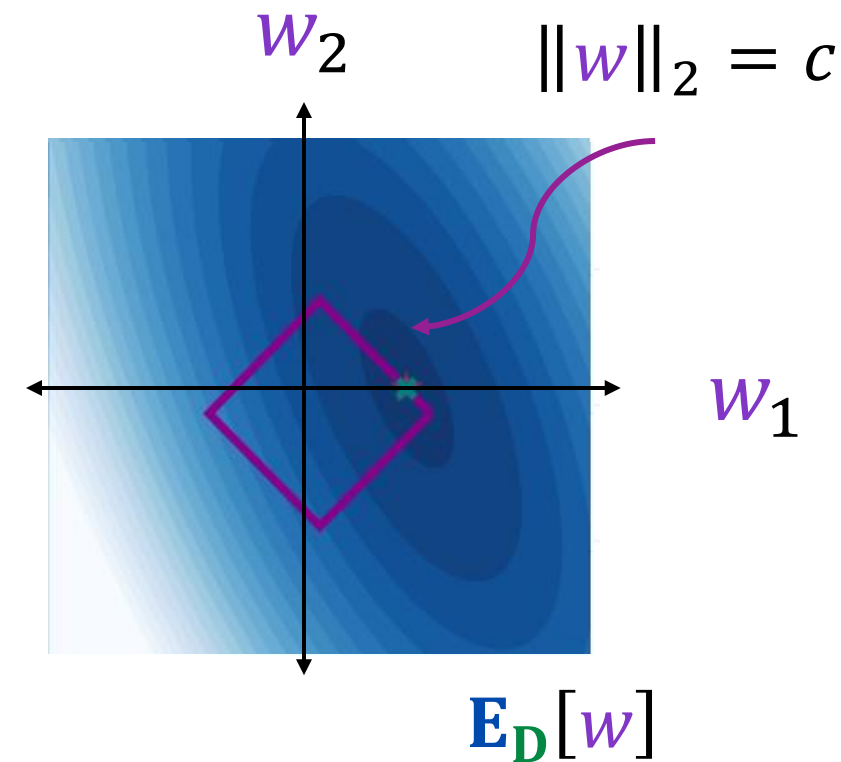
$$\frac{1}{2} \sum_{n=1}^N (\mathbf{t}_n - y(\mathbf{x}_n, \mathbf{w}))^2$$

$$\frac{1}{2} L_1[\mathbf{w}] = \frac{1}{2} \sum_d |\mathbf{w}_d|$$

$$\frac{1}{2} (\|\mathbf{t}_n - y(\mathbf{x}_n, \mathbf{w})\|_2)^2$$

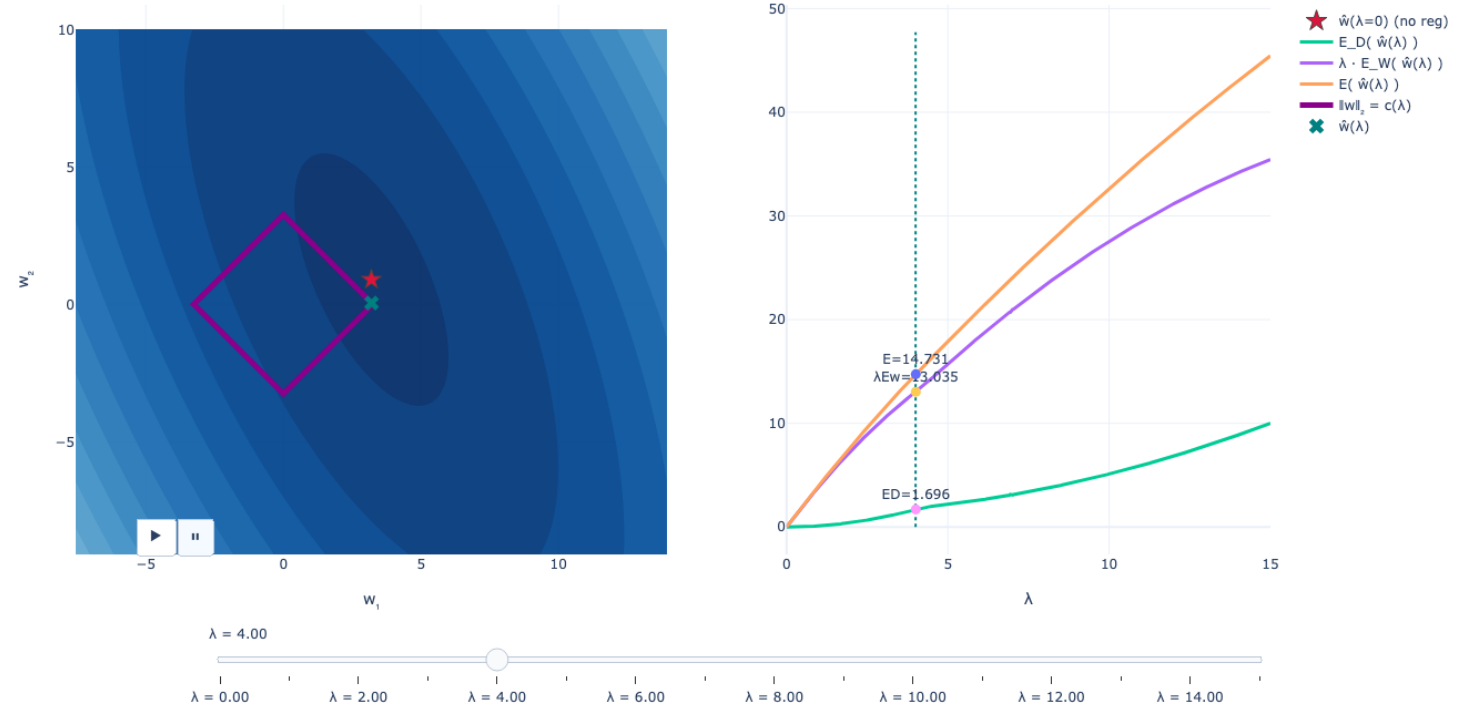
$$\frac{1}{2} (\|e_n\|_2)^2$$

length of the
residual vector



L2 Regularization Demo

[Interactive link](#)

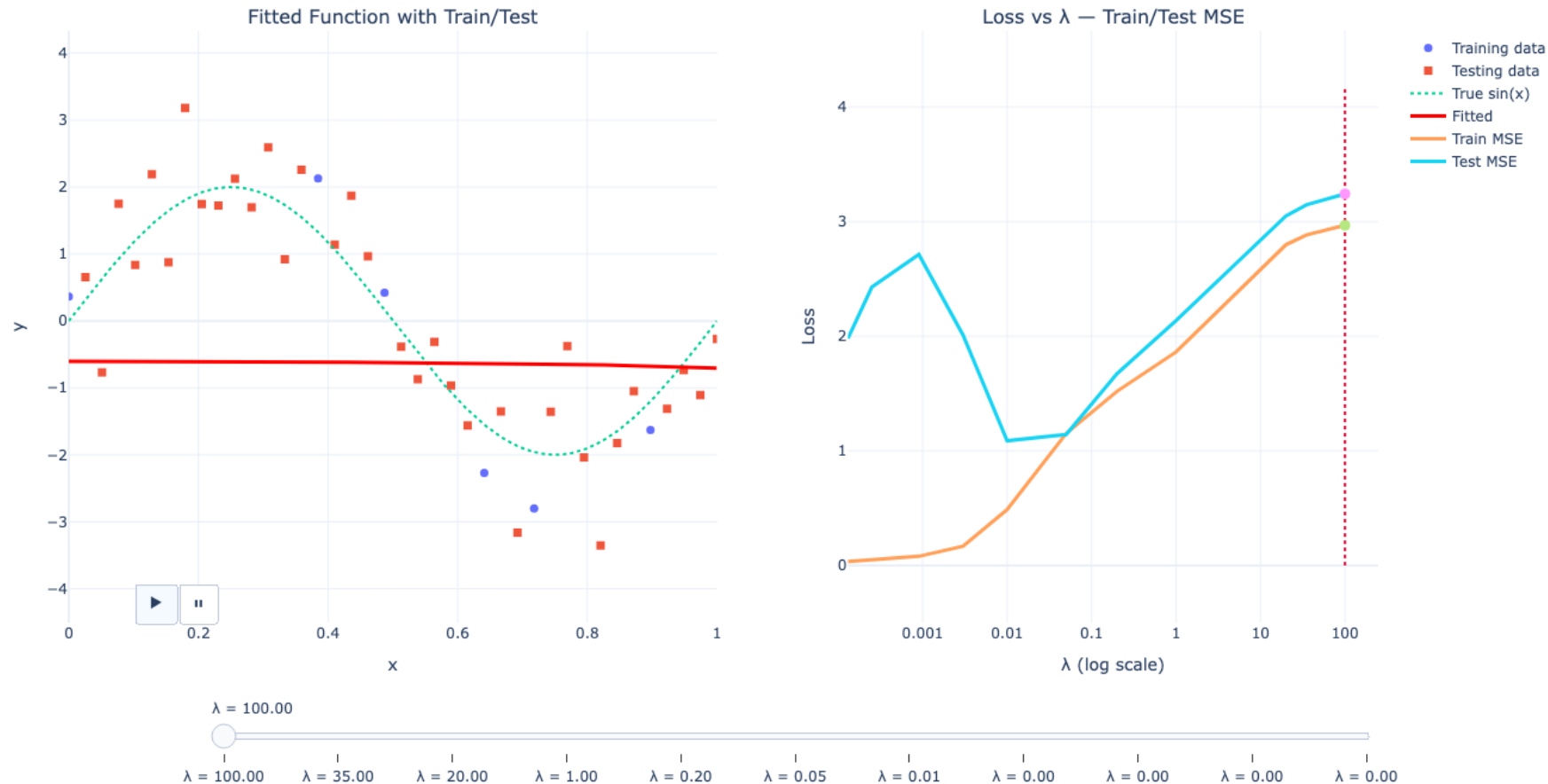


Impact of Regularization - Ridge



6101346

- λ controls complexity and helps avoid overfitting.



[Interactive link](#)

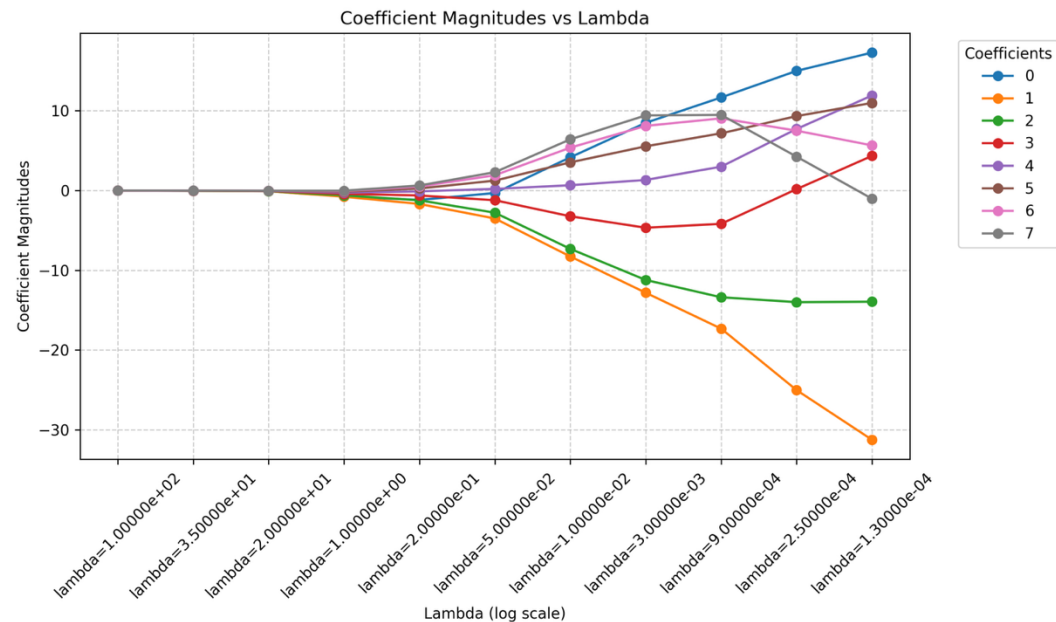
Impact of Regularization - Ridge



6101346

- λ controls complexity and helps avoid overfitting.

	lambda=1.00000e+02	lambda=3.50000e+01	lambda=2.00000e+01	lambda=1.00000e+00	lambda=2.00000e-01	lambda=5.00000e-02	lambda=1.00000e-02	lambda=3.00000e-03	lambda=9.00000e-04	lambda=2.50000e-04	lambda=1.30000e-04
Polynomial Degree											
0	-0.018966	-0.052435	-0.088490	-0.750335	-1.178313	-0.318366	4.162396	8.485826	11.662871	14.992023	17.289582
1	-0.019036	-0.052632	-0.088828	-0.777657	-1.686689	-3.508704	-8.271798	-12.817502	-17.319566	-25.023898	-31.248832
2	-0.016455	-0.045379	-0.076367	-0.611629	-1.230065	-2.784778	-7.316610	-11.216560	-13.384204	-13.996051	-13.938533
3	-0.013521	-0.037154	-0.062269	-0.426514	-0.623082	-1.219114	-3.232832	-4.667422	-4.177037	0.170150	4.339540
4	-0.010943	-0.029948	-0.049953	-0.273166	-0.113433	0.200421	0.657859	1.322626	2.988265	7.711474	11.909930
5	-0.008863	-0.024148	-0.040075	-0.159327	0.251802	1.238814	3.527585	5.538996	7.166978	9.322972	10.980703
6	-0.007234	-0.019625	-0.032402	-0.079540	0.488914	1.910190	5.373464	8.096688	9.045151	7.495621	5.653301
7	-0.005969	-0.016127	-0.026494	-0.025775	0.628286	2.293662	6.412034	9.413875	9.471402	4.241110	-1.029883

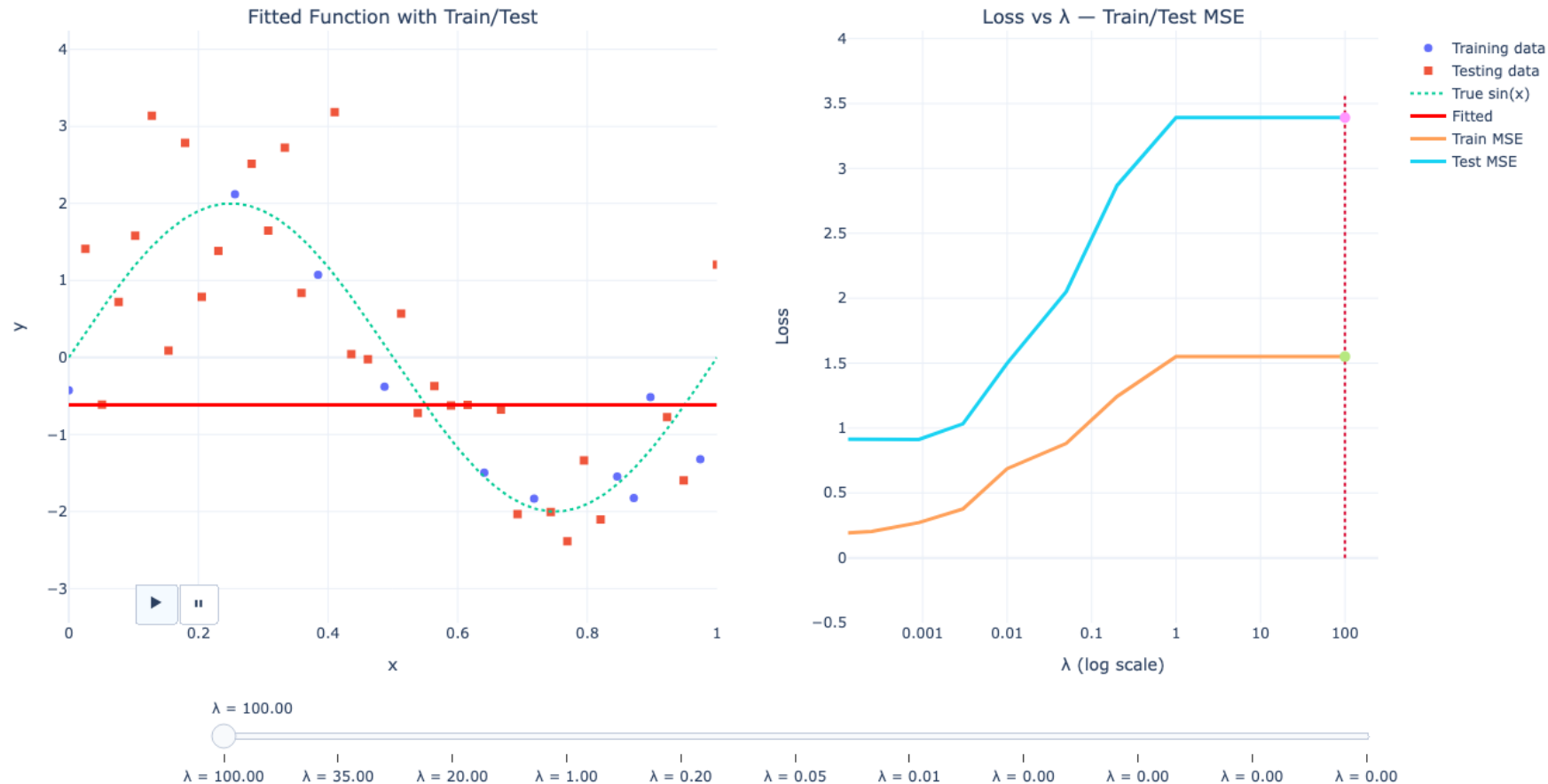




6101346

Impact of Regularization - Lasso

- λ controls complexity and helps avoid overfitting.



[Interactive link](#)

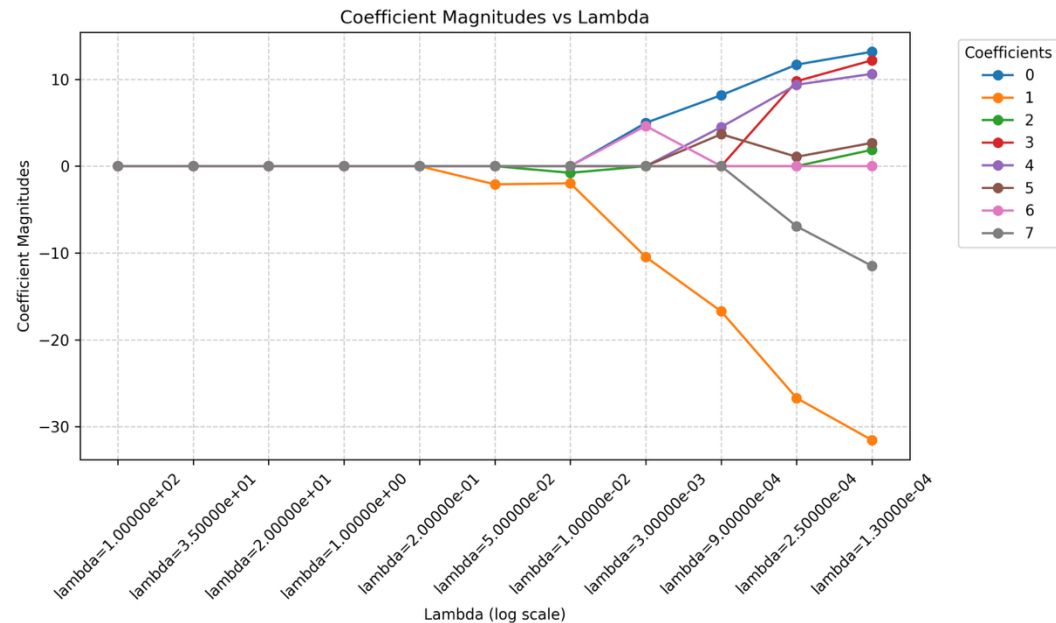
Impact of Regularization - Lasso



6101346

- λ controls complexity and helps avoid overfitting.

	lambda=1.00000e+02	lambda=3.50000e+01	lambda=2.00000e+01	lambda=1.00000e+00	lambda=2.00000e-01	lambda=5.00000e-02	lambda=1.00000e-02	lambda=3.00000e-03	lambda=9.00000e-04	lambda=2.50000e-04	lambda=1.30000e-04
Polynomial Degree											
0	-0.0	-0.0	-0.0	-0.0	-0.0	-0.000000	0.000000	4.986034	8.175296	11.697458	13.179180
1	-0.0	-0.0	-0.0	-0.0	-0.0	-2.095965	-1.975668	-10.457935	-16.722823	-26.709688	-31.561898
2	-0.0	-0.0	-0.0	-0.0	-0.0	-0.000000	-0.767903	-0.000000	-0.000000	0.000000	1.888739
3	-0.0	-0.0	-0.0	-0.0	-0.0	-0.000000	-0.000000	0.000000	0.000000	9.784207	12.196142
4	-0.0	-0.0	-0.0	-0.0	-0.0	-0.000000	-0.000000	0.000000	4.504546	9.382764	10.631082
5	-0.0	-0.0	-0.0	-0.0	-0.0	-0.000000	0.000000	0.000000	3.684894	1.077110	2.682036
6	-0.0	-0.0	-0.0	-0.0	-0.0	-0.000000	0.000000	4.632838	0.000000	-0.000000	-0.000000
7	-0.0	-0.0	-0.0	-0.0	-0.0	-0.000000	0.000000	0.000000	0.000000	-6.925711	-11.495779



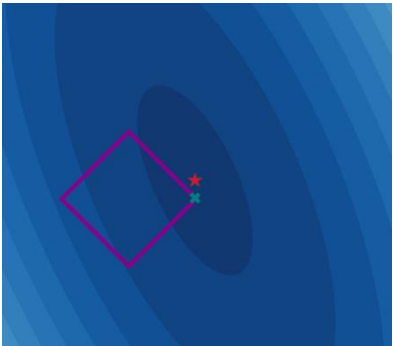
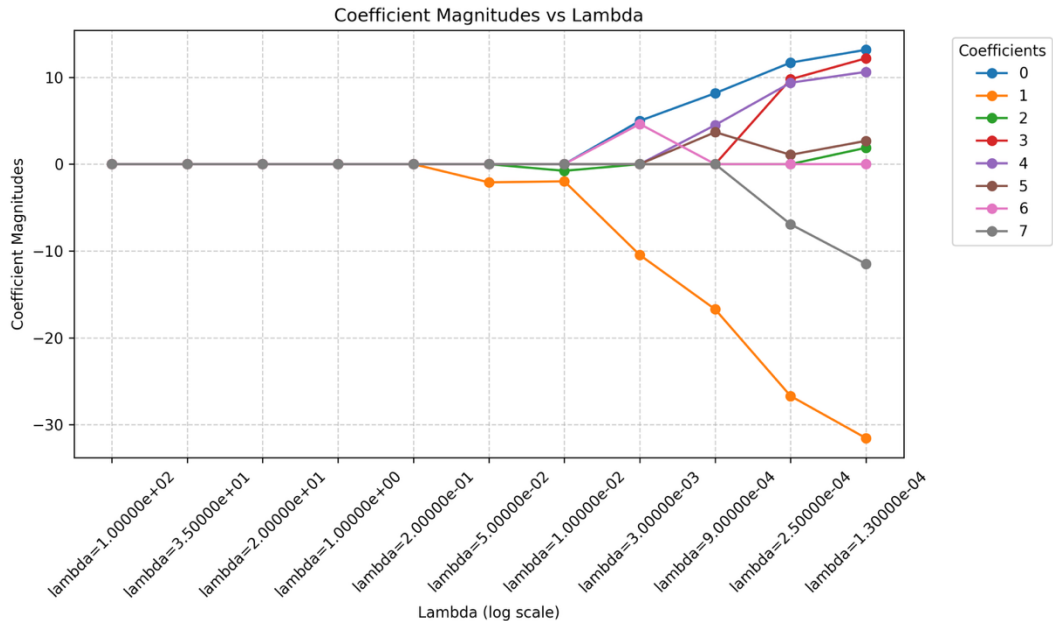


6101346

Impact of Regularization - Lasso

- λ controls complexity and helps avoid overfitting.

	lambda=1.00000e+02	lambda=3.50000e+01	lambda=2.00000e+01	lambda=1.00000e+00	lambda=2.00000e-01	lambda=5.00000e-02	lambda=1.00000e-02	lambda=3.00000e-03	lambda=9.00000e-04	lambda=2.50000e-04	lambda=1.30000e-04
Polynomial Degree											
0	-0.0	-0.0	-0.0	-0.0	-0.0	-0.000000	0.000000	4.986034	8.175296	11.697458	13.179180
1	-0.0	-0.0	-0.0	-0.0	-0.0	-2.095965	-1.975668	-10.457935	-16.722823	-26.709688	-31.561898
2	-0.0	-0.0	-0.0	-0.0	-0.0	-0.000000	-0.767903	-0.000000	-0.000000	0.000000	1.888739
3	-0.0	-0.0	-0.0	-0.0	-0.0	-0.000000	-0.000000	0.000000	0.000000	9.784207	12.196142
4	-0.0	-0.0	-0.0	-0.0	-0.0	-0.000000	-0.000000	0.000000	4.504546	9.382764	10.631082
5	-0.0	-0.0	-0.0	-0.0	-0.0	-0.000000	0.000000	0.000000	3.684894	1.077110	2.682036
6	-0.0	-0.0	-0.0	-0.0	-0.0	-0.000000	0.000000	4.632838	0.000000	-0.000000	-0.000000
7	-0.0	-0.0	-0.0	-0.0	-0.0	-0.000000	0.000000	0.000000	0.000000	-6.925711	-11.495779



Lasso
Least Absolute Shrinkage and
Selection Operator

When Normal Equation Gets Tricky

- Error Minimization
- Geometric Interpretation
- Evaluation
- Regularized Least Squares
- When Normal Equation Gets Tricky



6101346



6101346

When Normal Equation Gets Tricky

- When is $\mathbf{X}^T \mathbf{X}$ invertible?
 - Full column rank: $\text{rank}(\mathbf{X}) = D + 1$
 - Requires $N \geq D + 1$ (at least as many samples as features)
 - No perfect collinearity: $\det(\mathbf{X}^T \mathbf{X}) \neq 0$
 - All singular values be non-zero: $\sigma_1, \sigma_2, \dots, \sigma_{D+1} \geq 0$
- Illconditioning (numerical instability)
 - Even if $\det(\mathbf{X}^T \mathbf{X}) \neq 0$
 - **Condition number** $\kappa = \frac{\sigma_{\max}}{\sigma_{\min}} \gg 10^8$ small noise in the data creates huge weight swing
 - Why?
 - Rounding error will be amplified when calculating $\kappa \epsilon_{\text{machine}}$.



If we plug in the SVD of X , what is the simplified expression for the solution to the normal equation?



6101346

Fix #1: Ridge Trick

Stabilize with $(\mathbb{X}^T \mathbb{X} + \lambda I) \mathbf{w} = \mathbb{X}^T \mathbf{t}$

- $\lambda > 0$ boosts all singular values $\rightarrow$ lower κ .
- Closed-form solution: $\mathbf{w}^* = (\mathbb{X}^T \mathbb{X} + \lambda I)^{-1} \mathbb{X}^T \mathbf{t}$

Illconditioning Example

With the condition factor that is large, due to very small eigenvalues resulting from very similar features, the weights become unstable.

Try changing the noise and you will see substantial swings in weights.

```
N, D = 500, 2 # one bias + 2 features
X = np.ones((N, D + 1)) # first column = bias
x1 = np.random.randn(N)
x2 = x1 + 1e-4 * np.random.randn(N) # almost identical feature
X[:, 1] = x1
X[:, 2] = x2
```

Condition $(\mathbf{X}^T \mathbf{X}) =$
420140223.27925146

```
t = 3 + 2*x1 - 1*x2 + 0.1*np.random.randn(N) # targets
```

```
XtX = X.T @ X
Xty = X.T @ t
w_ne = np.linalg.solve(XtX, Xty)
```

```
lam = 1e-2
w_ridge = np.linalg.solve(XtX + lam*np.eye(D+1), Xty)
```

```
w_neq: [ 3.00273423 12.87110771 -11.87562795]
```

```
w_ridge : [3.00275573 0.50070697 0.49481053]
```



6101346

Fix #2: Moore–Penrose Pseudo-Inverse

- Singular Value Decomposition (SVD) recipe:
 - $X = U\Sigma V^T$ where $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_r)$ with $r = \text{rank}(X)$.
 - Every real matrix can be factorized this way; U and V have orthogonal columns ($U^T U = V^T V = I$), Σ holds the “stretch factors”.

- Moore–Penrose Pseudo-inverse:

$$\Sigma_{ii}^+ = \begin{cases} \frac{1}{\sigma_i} & \text{if } \sigma_i > \varepsilon \sigma_{\max} \\ 0 & \text{otherwise} \end{cases}$$

- We safely “flip” only the non-tiny singular values.

$$X^+ = V\Sigma^+ U^T$$

- Penrose conditions: $XX^+X = X$, $X^+XX^+ = X^+$, $(XX^+)^T = XX^+$, $(X^+X)^T = X^+X$. These make X^+ unique.



6101346

Fix #2: Moore–Penrose Pseudo-Inverse

How can Penrose Pseudo-Inverse help us with solving normal equation?

- Moore–Penrose Pseudo-inverse:

$$\Sigma_{ii}^+ = \begin{cases} \frac{1}{\sigma_i} & \text{if } \sigma_i > \varepsilon \sigma_{max} \\ 0 & \text{otherwise} \end{cases}$$

- We safely “flip” only the non-tiny singular values.

Closed-form solution for least squares:

$$\mathbf{w} = \mathbf{X}^+ \mathbf{t}$$

This works no matter whether $\mathbf{X}$ is tall ($N > D$), square, or wide ($N < D$). Also regardless of singular values.



6101346

Fix 2: Why It Helps?

Issue	How pseudo-inverse fixes it?
Rank deficiency (perfect multicollinearity)	Sets 0 for $\sigma = 0 \Rightarrow$ removes redundant directions, gives the <i>minimum-norm</i> weight vector.



6101346

Fix 2: Why It Helps?

Issue	How pseudo-inverse fixes it?
Rank deficiency (perfect multicollinearity)	Sets 0 for $\sigma = 0 \Rightarrow$ removes redundant directions, gives the <i>minimum-norm</i> weight vector.
Ill-conditioning	Truncation threshold discards numerically meaningless directions, implicitly adds ridge-like regularization.



6101346

Fix 2: Why It Helps?

Issue	How pseudo-inverse fixes it?
Rank deficiency (perfect multicollinearity)	Sets 0 for $\sigma = 0 \Rightarrow$ removes redundant directions, gives the <i>minimum-norm</i> weight vector.
Ill-conditioning	Truncation threshold discards numerically meaningless directions, implicitly adds ridge-like regularization.
Underdetermined systems ($N < D$)	Returns the unique solution with the smallest L_2 -norm among infinitely many exact fits.



6101346

Fix 2: Why It Helps?

Issue	How pseudo-inverse fixes it?
Rank deficiency (perfect multicollinearity)	Sets 0 for $\sigma = 0 \Rightarrow$ removes redundant directions, gives the <i>minimum-norm</i> weight vector.
Ill-conditioning	Truncation threshold discards numerically meaningless directions, implicitly adds ridge-like regularization.
Underdetermined systems ($N < D$)	Returns the unique solution with the smallest L_2 -norm among infinitely many exact fits.
Implementation convenience	<code>np.linalg.pinv</code> & <code>torch.pinverse</code> expose <code>rcond</code> so you control the σ -cut-off in one line of code.



6101346

Fix 3: Sequential Learning

- In practice, we use sequential algorithms, also known as online algorithms, in which the data points are considered one at a time and the model parameters updated after each such presentation.
- **Algorithm:** Stochastic gradient descent, also known as sequential gradient descent:
 - If we assume that the error function is calculated based on all samples, then $E = \sum_n E_n$.
 - After presentation of datapoint n , the gradient descent algorithm updates parameters using $w^{(\tau+1)} = w^{(\tau)} - \eta \nabla E_n$

$$w^{(\tau+1)} = w^{(\tau)} - \eta \left(t_n - w^{(\tau)T} \phi(x_n) \right) \phi(x_n)$$

Known as Least
Mean Squares or
LMS

Learning rate

Lecture 7

Linear Regression (2)

Credit: Joseph E. Gonzalez and Narges Norouzi

Reference Book Chapters: Chapter 1.2, Chapter 4.[1.4-1.6]